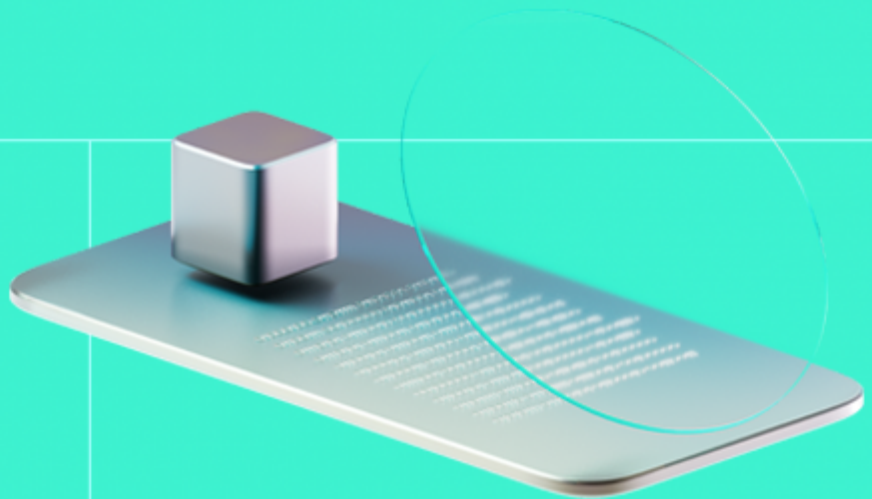




Smart Contract Code Review And Security Analysis Report

Customer: Dexalot

Date: 21/04/2026



We express our gratitude to the Dexalot team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Dexalot is an omni-chain, ERC20-less decentralized exchange protocol that operates across multiple EVM-compatible chains including Avalanche C-Chain, Dexalot L1, Arbitrum, Base, and Gunzilla.

Document

Name	Smart Contract Code Review and Security Analysis Report for Dexalot
Audited By	Olesia Bilenka, Ivan Bondar
Approved By	Khrystyna Tkachuk
Website	https://dexalot.com/
Changelog	16/02/2026 - Preliminary Report
	21/04/2026 - Final Report
Platform	Avalanche C-Chain; Dexalot L1; Arbitrum; Base; Gunzilla
Language	Solidity
Tags	DEX; Bridge; Upgradable; Centralization; OrderBook; Vault
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Dexalot/contracts/
Commit	396ddde
Final Commit	993df50

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

59

Total Findings

40

Resolved

3

Accepted

16

Mitigated

Findings by Severity

Severity

Count

Critical	1
High	0
Medium	5
Low	29

Vulnerability	Severity	Status
F-2026-14877 - Inverted isDirect Flag Logic Causes ERC20 Tokens to Not Be Collected in Direct Swaps	Critical	Fixed
F-2026-14844 - Untrusted Contract Remains Callable via Whitelisted Function After Trust Revocation	Medium	Mitigated
F-2026-14861 - Front-Running DoS on Batch Settlement via Rolling Hash Invalidation	Medium	Fixed
F-2026-14863 - Missing Gas Tank Auto-Fill for Recipients in Bulk Transfers	Medium	Fixed
F-2026-14935 - Auction Mode Bypass in bulkTransferTokens Allows Transfers During Active Auctions	Medium	Fixed
F-2026-14991 - depositTokenFromContract Cannot Pay Bridge Fees	Medium	Fixed
F-2026-14835 - Denial of Service via Global Pending Request Limit Exhaustion	Low	Mitigated
F-2026-14837 - Vault Status Management Allows Setting NONE/DEPRECATED Status, Locking User Funds	Low	Fixed
F-2026-14838 - Unrestricted Share Minting by Settler Role Enables Value Extraction from Depositors	Low	Fixed
F-2026-14840 - Unrestricted Withdrawal Amounts by Settler Role Enables Value Extraction from Withdrawing Users	Low	Fixed
F-2026-14841 - Missing Token Validation in Withdrawal Settlement Allows Arbitrary Asset Dispatch	Low	Fixed
F-2026-14845 - fallback Ignores ContractAccess Type, Allowing Function Execution on Contracts With Restricted Access Levels	Low	Mitigated

Vulnerability	Severity	Status
F-2026-14846 - No Mechanism to Remove Whitelisted Functions in OmniVaultExecutor	Low	Fixed
F-2026-14850 - First Batch Vulnerable to Immediate Unwind	Low	Fixed
F-2026-14869 - TRUSTED_TRANSFER_ROLE Allows Arbitrary Fund Transfers	Low	Mitigated
F-2026-14874 - Strict Payload Length Validation May Reject In-Flight Messages During Upgrade	Low	Mitigated
F-2026-14875 - Unvalidated RFQ Address from Cross-Chain Payload Enables Potential Fund Misdirection	Low	Mitigated
F-2026-14878 - Removal of Pause Functionality Eliminates Emergency Response Capability	Low	Mitigated
F-2026-14879 - Immutable swapSigner Prevents Key Rotation and Creates Irrecoverable Compromise Risk	Low	Mitigated
F-2026-14880 - Silent Failure in matchAuctionOrder Causes Infinite Loop in matchAuctionOrders	Low	Fixed
F-2026-14882 - Removal of Swap Expiry Mechanism Prevents Server-Side Quote Invalidation	Low	Mitigated
F-2026-14883 - Missing msg.value Validation for Native Token Swaps	Low	Fixed
F-2026-14891 - Switching to bulkTransferTokens Removes AutoFill Functionality for Claim Recipients	Low	Fixed
F-2026-14896 - Use of Deprecated transfer Function for Native Token Withdrawal	Low	Fixed
F-2026-14900 - Semantic Storage Variable Change Requires Migration During InventoryManager Upgrade	Low	Mitigated
F-2026-14901 - Division by Zero When Withdrawing Total Remaining Inventory	Low	Fixed
F-2026-14905 - K Value Not Validated as Multiple of 4 Causes Incorrect Fee Calculation	Low	Fixed
F-2026-14907 - calculateWithdrawalFee Returns Zero Fee for Zero Inventory Instead of Reverting	Low	Fixed
F-2026-14910 - Unenforced Invariant Between minTradeAmount and minPostAmount Causes Unexpected Order Cancellations	Low	Mitigated
F-2026-14914 - Immutable trustedForwarder Without Revocation Capability	Low	Accepted
F-2026-14939 - Token Removal Migration Leaves Orphaned Data in tokenInfoMapBySymbolChainId	Low	Accepted
F-2026-14948 - Cross-Chain Message Loss During Contract Upgrade Due to Payload Format Incompatibility	Low	Mitigated
F-2026-15243 - Defensive Check Ordering in _calcSharesToMint Allows Theoretical Division by Zero	Low	Fixed
F-2026-16203 - [DualDefense] Permissionless UnwindBatch Can Dkip Batch Ordering And Permanently Lock Finalized Withdrawal Requests	Low	Accepted
F-2026-16204 - [DualDefense] finalizeBatch Can Never Finalize Batch Zero Because It Reads batchId - 1 Before The Zero-Batch Guard	Low	Fixed

Vulnerability	Severity	Status
F-2026-14842 - Missing tokenType Validation Allows Irreversible Token Invalidation	Info	Fixed
F-2026-14843 - Whitelisted Function Selector Mapping May Silently Overwrite Existing Entries	Info	Fixed
F-2026-14865 - Missing Event Emissions for Key State Changes in Vault Contracts	Info	Mitigated
F-2026-14870 - Missing NatSpec Documentation for Function Parameters in PortfolioSub	Info	Fixed
F-2026-14872 - Undocumented Zero Parameters in getNativeBridgeFee	Info	Fixed
F-2026-14884 - receive Function in DexalotRouter Accepts ETH Without Processing, Leading to Stuck Funds	Info	Fixed
F-2026-14885 - Missing Reentrancy Guards on Swap Functions Reduces Defense-in-Depth	Info	Fixed
F-2026-14886 - Missing Reentrancy Guard in multiPartialSwap and fallback Functions	Info	Mitigated
F-2026-14894 - Missing Explicit Check for Maximum Token Limit Results in Overflow Revert	Info	Fixed
F-2026-14895 - Missing Validation for Duplicate and Empty Token Symbols	Info	Fixed
F-2026-14897 - Missing Event Emissions for State-Changing Functions	Info	Mitigated
F-2026-14899 - Misleading Documentation and Dead Code in floor Function	Info	Fixed
F-2026-14904 - Missing Validation in setDisplayDecimals Can Cause DoS for Trade Pair	Info	Mitigated
F-2026-14912 - Inconsistent Documentation for Default Allowed Slippage Percent	Info	Fixed
F-2026-14915 - Native Tokens Lost During ERC20 Swaps Through Router	Info	Fixed
F-2026-14919 - Inconsistent Native Token Handling in sendTokenBalance Function	Info	Fixed
F-2026-14931 - Redundant Conditional Logic in decrement Function	Info	Fixed
F-2026-14932 - Gas Inefficiency in powMultipleOf4BPS	Info	Fixed
F-2026-14933 - Misleading NatSpec Comment in powMultipleOf4BPS Function	Info	Fixed
F-2026-14940 - Excess Native Token Not Refunded When User Pays Bridge Fee	Info	Fixed
F-2026-14941 - Off-by-One Error in PortfolioBridgeMain Contract Balance Check	Info	Fixed
F-2026-14985 - Inverted NatSpec Comment in _updateTakerFeeCollector	Info	Fixed
F-2026-16205 - [DualDefense] Duplicate-Heavy Deposit Batches Are Atomic, Cannot Be Partially Settled, and Block Follower Batches	Info	Fixed
F-2026-16206 - [DualDefense] Current Design Allows for maxPerDeposit To Be Bypassed via Same tokenIds in requestDeposit	Info	Fixed

Documentation quality

- Functional requirements are detailed.
 - Project overview is detailed
 - All roles in the system are described.
 - Use cases are described and detailed.
 - For each contract, all futures are described.
 - All interactions are described.
- Technical description is detailed.
 - Run instructions are provided.
 - Technical specification is provided.
 - The NatSpec documentation is sufficient.

Code quality

- The development environment is configured.

Test coverage

The project demonstrates a high level of automated test coverage. Due to the combined use of Foundry and Hardhat within the same codebase, a single consolidated coverage metric cannot be reliably derived.

- Deployment and basic user interactions are covered with tests.
- Negative cases coverage is present.
- Interactions by several users are tested.

Table of Contents

System Overview	8
Privileged Roles	9
Potential Risks	12
Findings	14
Vulnerability Details	14
Disclaimers	149
Appendix 1. Definitions	150
Severities	150
Potential Risks	150
Appendix 2. Scope	151
Appendix 3. Additional Valuables	153

System Overview

Dexalot is an omni-chain, ERC20-less decentralized exchange protocol that operates across multiple EVM-compatible chains including Avalanche C-Chain, Dexalot L1, Arbitrum, Base, and Gunzilla. The protocol implements a unique architecture where tokens are locked in mainnet Portfolio contracts and their virtual representations are traded on the Dexalot L1 using Central Limit Order Books (CLOB) without deploying ERC20 contracts on the L1.

The system supports multiple cross-chain bridges (ICM - Avalanche's Inter-Chain Messaging and LayerZero) for deposits and withdrawals, enabling users to deposit with one bridge and withdraw with another. The architecture implements gas abstraction through a GasStation contract that automatically swaps deposited tokens for native gas tokens, eliminating the need for users to manage gas tokens on Dexalot L1.

Dexalot also features a Request-For-Quote (RFQ) system for single-chain and cross-chain token swaps using off-chain price discovery from the Dexalot L1 orderbook. Additionally, the protocol introduces OmniVaults - managed vaults that allow automated trading strategies through whitelisted executor contracts.

The protocol aims to provide professional-grade trading with price-time priority orderbooks, maker/taker fee structures with volume-based rebates, auction mechanisms for token launches, and sophisticated inventory management across chains.

Files in Scope

Core Trading Contracts:

- **TradePairs.sol** - Manages trade pairs with Central Limit Order Books, handling order creation, matching, cancellation, and execution with support for LIMIT/MARKET orders, auction modes, and self-trade prevention (STP).
- **OrderBooks.sol** - Implements Red-Black-Tree based orderbooks with price-time priority, maintaining buy/sell books for each trade pair.
- **PortfolioSub.sol** - Dexalot L1 portfolio managing virtual token balances, trade executions, fee calculations, gas abstraction (AutoFill), and withdrawal processing.
- **PortfolioSubHelper.sol** - Helper contract managing rate overrides, volume-based rebates, admin accounts for rates, and taker fee collectors.

Mainnet Contracts:

- **PortfolioMain.sol** - Gateway for deposits to Dexalot L1, processes withdrawal messages, maintains ERC20 token mappings, and handles trusted contracts.
- **Portfolio.sol** - Abstract base contract for portfolio functionality shared between PortfolioMain and PortfolioSub.

Bridge Contracts:

- **PortfolioBridgeMain.sol** - Bridge aggregator on mainnets supporting multiple bridge providers (ICM, LayerZero), handling cross-chain message encoding/decoding.

- **PortfolioBridgeSub.sol** - Dexalot L1 bridge aggregator with symbol mapping, inventory management integration, delayed transfers, and dynamic bridge fee calculations.
- **DefaultBridgeApp.sol** - Abstract base implementation for bridge providers with remote chain configuration.
- **ICMApp.sol** - Avalanche ICM bridge implementation using Teleporter for inter-chain messaging.

RFQ Contracts:

- **MainnetRFQ.sol** - Upgradeable RFQ contract for same-chain and cross-chain swaps with signed quotes, slippage protection, and swap queuing.
- **DexalotRFQ.sol** - Non-upgradeable RFQ contract with immutable swap signer and wrapped native token configuration.
- **DexalotRouter.sol** - Router contract facilitating aggregator swaps via RFQ order execution with multi-hop swap support.

OmniVault Contracts:

- **OmniVaultManager.sol** - Manages OmniVaults, handling deposit/withdrawal requests in batches, vault registration, and share token operations.
- **OmniVaultCreator.sol** - Facilitates vault creation with initial deposits, fee collection, and request lifecycle management.
- **OmniVaultExecutor.sol** - Executor contract allowing EOA trading bots to interact with whitelisted functions on trusted contracts.
- **OmniVaultExecutorSub.sol** - Dexalot L1 executor for dispatching assets from OmniVaults back to users.
- **OmniVaultShare.sol** - ERC20 vault share tokens with LayerZero OFT cross-chain functionality.
- **OmniVaultRegistry.sol** - Registry for OmniVault contracts and configurations.

Supporting Contracts:

- **DelayedTransfers.sol** - Implements withdrawal delays and volume caps for security on large transfers.
- **InventoryManager.sol** - Manages multi-chain token inventory, calculates withdrawal fees based on liquidity distribution.

Libraries:

- **UtilsLibrary.sol** - Utility functions for byte conversions, decimal handling, and auction mode checks.
- **InventoryFeeCalculatorLibrary.sol** - Calculates inventory-based withdrawal fees using invariant formulas.

Privileged roles

- **TradePairs.sol**
 - **DEFAULT_ADMIN_ROLE**: Can pause/unpause contract, pause/unpause individual trade pairs, add/remove order types, set auction mode/price, set min/max trade amounts, set maker/taker rates, set display decimals, set slippage percent, remove trade pairs, and execute unsolicited cancels.
 - **EXCHANGE_ROLE**: Can add new trade pairs.
- **PortfolioSub.sol**

- **DEFAULT_ADMIN_ROLE** : Can add/remove tokens, set fee address, set gas station, set treasury, set portfolio minter, set portfolio bridge, set portfolio sub helper, set auction mode, pause/unpause contract, pause deposits, and set bridge parameters.
- **EXECUTOR_ROLE** : Can execute trades (**addExecution**), adjust available balances, set auction mode, and trigger autoFill gas operations.
- **PORTFOLIO_BRIDGE_ROLE** : Can process cross-chain transfer payloads (deposits).
- **TRUSTED_TRANSFER_ROLE** : Can perform bulk transfers on behalf of other users.
- **PortfolioMain.sol**
 - **DEFAULT_ADMIN_ROLE** : Can add/remove tokens, set banned accounts, add/remove trusted contracts, collect bridge fees, set wrapped native, set min deposit multiplier, pause/unpause, and set bridge parameters.
 - **PORTFOLIO_BRIDGE_ROLE** : Can process cross-chain transfer payloads (withdrawals).
- **Portfolio.sol** (Base)
 - **DEFAULT_ADMIN_ROLE** : Can set portfolio bridge, pause/unpause, pause deposits, set bridge parameters, and set L1 decimals.
 - **PORTFOLIO_BRIDGE_ROLE** : Can set bridge parameters.
- **PortfolioBridgeMain.sol**
 - **DEFAULT_ADMIN_ROLE** : Can enable/disable bridge providers, set trusted remote addresses, set default destination chain, set user pays fee settings, set portfolio, set gas airdrop, refund native balance, and enable cross-chain swap destinations.
 - **BRIDGE_USER_ROLE** : Can pause/unpause, send cross-chain messages (held by **Portfolio** and **MainnetRFQ**).
 - **BRIDGE_ADMIN_ROLE** : Can set bridge parameters on portfolio.
 - **BRIDGE_PROVIDER_ROLE** : Can process incoming cross-chain payloads (held by bridge app contracts).
- **PortfolioBridgeSub.sol**
 - **DEFAULT_ADMIN_ROLE** : Same as **PortfolioBridgeMain** plus set delayed transfers, set inventory manager, set bridge fee multiplier, set max bridge fee caps, set option gas costs, and set L1 decimals.
 - **BRIDGE_USER_ROLE** : Can add/remove tokens, send cross-chain messages.
 - **BRIDGE_ADMIN_ROLE** : Can set bridge fees, execute delayed transfers.
 - **BRIDGE_PROVIDER_ROLE** : Can process incoming cross-chain payloads.
- **DexalotRFQ.sol**
 - **DEFAULT_ADMIN_ROLE** : Can set portfolio bridge, set portfolio main, add/remove admins, add/remove rebalancers, add/remove volatility admins.
 - **REBALANCER_ADMIN_ROLE** : Can claim/withdraw assets, send token balance, receive native tokens.
 - **PORTFOLIO_BRIDGE_ROLE** : Can process cross-chain trade payloads.
 - **VOLATILITY_ADMIN_ROLE** : Can set slippage points.
- **DexalotRouter.sol**
 - **DEFAULT_ADMIN_ROLE** : Can set allowed RFQ contracts and retrieve tokens.
- **OrderBooks.sol**
 - **DEFAULT_ADMIN_ROLE** : Can set trade pairs contract.
 - **EXECUTOR_ROLE** : Can add orderbooks, add/remove orders, match trades (held by **TradePairs**).
- **DelayedTransfers.sol**
 - **DEFAULT_ADMIN_ROLE** : Can check thresholds, execute delayed transfers, set delay thresholds, set delay period, set epoch length, set epoch volume caps, update volume.

- **InventoryManager.sol**
 - **DEFAULT_ADMIN_ROLE**: Can update **PortfolioBridgeSub**, set scaling factors, remove scaling factors, update future **K** value, update **K** value.
 - **PORTFOLIO_BRIDGE_ROLE**: Can increment/decrement inventory, remove tokens.
- **PortfolioSubHelper.sol**
 - **DEFAULT_ADMIN_ROLE**: Can set min taker rate, add/remove admin accounts for rates, add/remove volume based rebates, add/remove rate overrides, set taker fee collectors.
- **OmniVaultManager.sol**
 - **DEFAULT_ADMIN_ROLE**: Can register vaults, pause/unpause vaults, update vault details, add/update token details, pause/unpause contract, set portfolio.
 - **SETTLER_ROLE**: Can bulk settle deposit and withdrawal requests.
- **OmniVaultCreator.sol**
 - **DEFAULT_ADMIN_ROLE**: Can accept and fund vaults, reject vault requests, set fee token, set fee amount, set reclaim delay, withdraw collected fees.
- **OmniVaultExecutor.sol / OmniVaultExecutorSub.sol**
 - **DEFAULT_ADMIN_ROLE**: Can set whitelisted functions, set trusted contracts, set portfolio.
 - **OMNITRADER_ROLE**: Can execute whitelisted functions via fallback, send native, approve tokens, dispatch assets.
- **OmniVaultShare.sol**
 - **owner**: Can set **OmniVaultManager**, configure **LayerZero** settings.
- **ICMApp.sol**
 - **owner**: Can set portfolio bridge, set relayers, add/clear relayers, set gas limits, set native bridge fees.
- **DefaultBridgeApp.sol**
 - **portfolioBridge** (modifier): Can send messages and set remote chains.

Potential Risks

- **Scope Definition and Security Guarantees:** The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **Upgradability and Future Version Modifications:** The contract's upgradable nature means that its implementation can be modified in future versions. This flexibility allows for necessary updates and improvements but also introduces uncertainty for users regarding future changes in the contract's behavior and rules. Users should stay informed about any contract upgrades and understand their implications.
- **Cross-Chain Communication Risks:** Operating across multiple chains (Avalanche, Ethereum, Arbitrum, Gunzilla) introduces risks associated with cross-chain communication and bridging. This includes potential delays, miscommunications, or exploits that target the bridge mechanisms.
- **Governance and Access Control Risks:** The system relies heavily on role-based access control for administrative tasks. Mismanagement of these roles or exploitation of these privileges could lead to unauthorized actions, including pausing key functionalities or misconfiguring critical parameters.
- **Unbounded Loops in TradePairs Contract:** The TradePairs contract contains unbounded loops, which can result in transactions running out of gas when processing large numbers of orders. This could lead to failed transactions and wasted gas. Users, particularly market makers, should be mindful of the potential for gas exhaustion when submitting multiple orders in a single transaction.
- **Inventory-Based Withdrawal Limitations:** There's a risk associated with user deposits not being withdrawable to the originally deposited chain if the inventory on that chain drops significantly. This could lead to situations where users are unable to access their funds on their preferred chain, potentially causing liquidity and user experience issues.
- **Maximum Fee Cap:** Users are advised to remain aware of the potential for high withdrawal fees, particularly in scenarios where inventory is low on their desired withdrawal chain. They should also pay attention to frontend notifications and consider alternative options provided by the platform to minimize fees. Users should stay informed about any future updates or changes to the fee structure and amplification coefficient management.
- **Unpredictable Withdrawal Fees Due to Sensitivity Exponent (K) and Inventory Fluctuations:** The sensitivity exponent K (range 8-32) and per-token target ratios can be adjusted by the protocol, influencing withdrawal fees that range from 0.01% to 5%. Fees are calculated dynamically based on real-time inventory distribution across chains, meaning users may face variable costs depending on current inventory imbalance at withdrawal time. While users who deposited from a specific chain receive fee exemption up to their deposited amount, others should remain aware that fees fluctuate based on inventory state and configured parameters.
- **Elevated Privileges of Trusted Contracts in Dexalot's Ecosystem:** Trusted Contracts, particularly those used for auction coordination, are granted significant powers, including the ability to deposit tokens on behalf of users. While these contracts are currently limited to auction-related activities, the broad privileges they hold could lead to unintended consequences if compromised or misused. Though Dexalot ensures these contracts are used specifically for auction activities, users should be aware of the elevated control these contracts possess and the reliance on their security.

- **Decimal Variation in Cross-Chain Transactions:** The protocol is designed to support tokens with differing decimal places across various blockchains. The protocol takes on the responsibility of correctly setting, validating, and managing these decimal variations to ensure accurate value scaling during cross-chain transfers and fee calculations. This approach introduces additional complexity, particularly in the truncation and scaling logic, but significantly increases flexibility and token supportability across networks.
- **Upgrade Routine Risk:** Inserting new variables or changing types in structs used in mappings can alter the storage layout. In some scenarios, differences in optimizer settings may modify struct packing or mapping lookups, inadvertently leading to data misalignment during an upgrade. Since the upgrade process is outside the audit scope, the project team must validate any upgrades in staging or test environments to ensure storage remains consistent and no data corruption occurs.

Findings

Vulnerability Details

[F-2026-14877](#) - Inverted isDirect Flag Logic Causes ERC20 Tokens to Not Be Collected in Direct Swaps - Critical

Description:

The `isDirect` flag logic in `_executeOrder` is inverted, causing the contract to skip collecting ERC20 tokens from users who call `simpleSwap` or `partialSwap` directly. The contract releases maker tokens to the user without receiving taker tokens in return, enabling drainage of pre-funded ERC20 liquidity.

In `_executeOrder`, the `isDirect` flag is set based on whether the sender equals the trusted forwarder:

```
SwapData memory swapData = SwapData({
    // ...
    msgSender: _sender,
    isDirect: _sender == trustedForwarder // BUG: false for direct calls
});
```

When a user calls `simpleSwap` directly, `_sender` is the user's address. Since the user is not the `trustedForwarder`, the condition evaluates to false, setting `isDirect = false`.

In `_takeFunds`, native token swaps are handled first and return early, but **ERC20** token transfers are only executed when `isDirect` is `true`:

```
function _takeFunds(SwapData memory _swapData) private {
    if (_swapData.srcAsset == address(0)) {
        require(msg.value >= _swapData.srcAmount, "RF-IMV-01");
        if (keepWrapped) {
            IWrappedToken(wrappedNative).deposit{value: _swapData.srcAmount}();
        }
        return;
    }

    if (_swapData.isDirect) {
        IERC20(_swapData.srcAsset).safeTransferFrom(_swapData.msgSender, address(this), _swapData.srcAmount);
    }

    if (!keepWrapped && _swapData.srcAsset == wrappedNative) {
```

```

        IWrappedToken(wrappedNative).withdraw(_swapData.srcAmount);
    }
}

```

When `isDirect` is `false`, the `safeTransferFrom` is skipped entirely. The function proceeds to `_releaseFunds` which sends maker tokens to the user without the contract having received any taker tokens.

Affected Functions:

Function	isDirect Value	Vulnerable?
simpleSwap (direct call)	false	Yes (ERC20 source)
partialSwap (direct call)	false	Yes (ERC20 source)
simpleSwap (via Router)	false	No (Router pre-transfers)
partialSwap (via Router)	false	No (Router pre-transfers)
xChainSwap	true (hardcoded)	No

Not Affected:

- Native token source swaps are protected by the `msg.value` check that occurs before the `isDirect` conditional
- `xChainSwap` hardcodes `isDirect: true` in `_executeXSwap`, so tokens are correctly pulled
- Router-based swaps work correctly because **DexalotRouter** transfers taker tokens before calling the RFQ contract

This leads to:

- **ERC20 liquidity drain:** Any user can call `simpleSwap` or `partialSwap` directly with a valid quote for an ERC20-to-ERC20 or ERC20-to-native swap, receive maker tokens without paying taker tokens, and repeat until pre-funded ERC20 liquidity is fully drained.
- **Immediate exploitability:** The vulnerability requires only a valid signed quote, which users legitimately obtain through normal protocol operation. The contract is pre-funded by rebalancers via `sendTokenBalance`, providing the liquidity that can be extracted.

It enables direct theft of all pre-funded ERC20 liquidity with no special privileges required.

Assets:

- contracts/DexalotRFQ.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Critical

Recommendations

Remediation: The `isDirect` logic should be corrected to properly identify when tokens need to be pulled from the user. For direct calls where the contract must collect **ERC20** tokens, `isDirect` should evaluate to `true`. The condition should be inverted to `isDirect: _sender != trustedForwarder` or the semantic meaning of the flag should be reconsidered to ensure token collection occurs for all direct user interactions with **ERC20** source assets.

Resolution: Fixed in `c206920`:

The change in **DexalotRFQ** updates `isDirect` from `_sender == trustedForwarder` to `_sender == msg.sender` inside `_executeOrder`.

This restores ERC20 collection behavior for direct `simpleSwap` and `partialSwap` calls because `_takeFunds` executes `safeTransferFrom` when `isDirect` is `true`.

The reported drain condition from direct ERC20 swaps is removed by this patch.

Evidences

Reproduce:

PoC Steps:

- Deploy **DexalotRFQ** with `swapSigner` → contract ready
- Fund RFQ with 1000 ALOT (rebalancer) → RFQ has ERC20 liquidity
- Attacker obtains a valid signed order for USDC→ALOT swap → valid quote acquired
- Attacker calls `simpleSwap` directly (no approval needed) → `isDirect=false`, ERC20 transfer skipped

PoC Code:

```
function test_isDirect_Inverted_DrainERC20Liquidity() public {
    uint256 makerAmount = 100e18; // ALOT to receive
    uint256 takerAmount = 50e6;   // USDC to pay (but won't)

    // Record initial balances
```



```

uint256 attackerAlotBefore = alot.balanceOf(attacker);
uint256 attackerUsdcBefore = usdc.balanceOf(attacker);
uint256 rfqAlotBefore = alot.balanceOf(address(rfq));
uint256 rfqUsdcBefore = usdc.balanceOf(address(rfq));

// Create valid signed order
IDexalotRFQ.Order memory order = IDexalotRFQ.Order({
    nonceAndMeta: (uint256(uint160(attacker)) << 96) | 1,
    expiry: uint128(block.timestamp + 3600),
    makerAsset: address(alot),
    takerAsset: address(usdc), // ERC20 source - vulnerable path
    maker: address(rfq),
    taker: attacker,
    makerAmount: makerAmount,
    takerAmount: takerAmount
});

bytes memory signature = _signOrder(order);

// Attacker does NOT approve USDC (doesn't need to due to bug)
vm.prank(attacker);
rfq.simpleSwap(order, signature);

// Verify exploit: attacker received ALOT without paying USDC
assertEq(alot.balanceOf(attacker), attackerAlotBefore + makerAmount, "Attacker did not receive ALOT");
assertEq(usdc.balanceOf(attacker), attackerUsdcBefore, "Attacker USDC was taken (unexpected)");
assertEq(alot.balanceOf(address(rfq)), rfqAlotBefore - makerAmount, "RFQ ALOT not reduced");
assertEq(usdc.balanceOf(address(rfq)), rfqUsdcBefore, "RFQ received USDC (unexpected)");

console.log("=== Exploit Successful ===");
console.log("Attacker ALOT gained:", makerAmount / 1e18, "ALOT");
console.log("Attacker USDC spent: 0 USDC");
console.log("RFQ ALOT drained:", makerAmount / 1e18, "ALOT");
}

```

Results:

Attacker receives 100 ALOT, pays 0 USDC → 100 ALOT drained

```

Ran 3 tests for test/DexalotRFQ_isDirectInverted.t.sol:IsDirectInvertedPoC
[PASS] test_isDirect_Inverted_DrainERC20Liquidity() (gas: 129774)

```

Logs:

```

=== Exploit Successful ===
Attacker ALOT gained: 100 ALOT
Attacker USDC spent: 0 USDC

```

```
RFQ ALOT drained: 100 ALOT
```

```
[PASS] test_isDirect_Inverted_RepeatedDrain() (gas: 367391)
```

```
Logs:
```

```
=== Full Drain Complete ===
```

```
Total ALOT stolen: 1000
```

```
[PASS] test_isDirect_Inverted_partialSwap_DrainERC20() (gas: 114351)
```

```
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 15.33ms (21.89ms  
CPU time)
```

```
Ran 1 test suite in 147.00ms (15.33ms CPU time): 3 tests passed, 0 failed, 0  
skipped (3 total tests)
```

Files:

DexalotRFQ_isDirectInverted.t.sol

[F-2026-14844](#) - Untrusted Contract Remains Callable via Whitelisted Function After Trust Revocation - Medium

Description:

The **OmniVaultExecutor** contract allows admin to revoke trust from a contract via `setTrustedContract(_contract, ContractAccess.NONE)`, but this does not remove the contract from the `whitelistedFunctions` mapping. As a result, the `OMNITRADER_ROLE` can still execute whitelisted functions on an untrusted contract through the `fallback` function, bypassing the intended access control.

The trust check is only performed when adding a whitelisted function:

```
function _setWhitelistedFunction(bytes4 _funcSignature, address _contract) internal {
    require(trustedContracts[_contract] != ContractAccess.NONE, "OT-IVTC-01");
    // Only checked here
    whitelistedFunctions[_funcSignature] = _contract;
}
```

However, the `fallback` function does not verify the current trust status:

```
fallback() external payable onlyRole(OMNITRADER_ROLE) {
    require(msg.data.length >= 4, "OT-FSNV-01");
    address impl = whitelistedFunctions[msg.sig];
    require(impl != address(0), "OT-FSNW-01"); // No trust check!

    (bool success, ) = impl.call{value: msg.value}(msg.data);
    _returnData(success);
}
```

When `setTrustedContract` is called to revoke trust, the whitelisted functions remain intact:

```
function setTrustedContract(address _contract, ContractAccess _access) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_contract != address(0), "OT-SAZ-01");
    trustedContracts[_contract] = _access; // No cleanup of whitelistedFunctions
}
```

Attack Scenario

1. Admin trusts **ContractA** with ERC20 access

```
trustedContracts[ContractA] = ERC20
```

2. Admin whitelists `doSomething(uint256)` on **ContractA**

```
whitelistedFunctions[0x12345678] = ContractA
```

3. **ContractA** is compromised or needs to be removed

Admin revokes trust:

```
trustedContracts[ContractA] = NONE
```

4. Admin EXPECTS **ContractA** is no longer callable, but:

```
whitelistedFunctions[0x12345678] = ContractA
```

5. **OMNITRADER** can still call **ContractA** via `fallback`

```
fallback → whitelistedFunctions[0x12345678] → ContractA → executed
```

This may lead to:

1. **Bypassed Access Control:** Untrusted contracts remain callable
2. **Security Assumption Violation:** Admin believes contract is disabled but it's not
3. **Potential Fund Loss:** If contract was untrusted due to compromise, funds may be at risk

Assets:

- contracts/vaults/OmniVaultExecutor.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status: Mitigated

Classification

Impact Rate: 5/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation:

Check trust status in `fallback`:

```
fallback() external payable onlyRole(OMNITRADER_ROLE) {
    require(msg.data.length >= 4, "OT-FSNV-01");
    address impl = whitelistedFunctions[msg.sig];
    require(impl != address(0), "OT-FSNW-01");
    require(trustedContracts[impl] != ContractAccess.NONE, "OT-CNUT-01");
    (bool success, ) = impl.call{value: msg.value}(msg.data);
    _returnData(success);
}
```

Clear whitelisted functions when revoking trust:

```
function clearWhitelistedFunctionsFor(
    address _contract,
    bytes4[] calldata _funcSignatures
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    for (uint256 i = 0; i < _funcSignatures.length; i++) {
        if (whitelistedFunctions[_funcSignatures[i]] == _contract) {
            delete whitelistedFunctions[_funcSignatures[i]];
        }
    }
}
```

Resolution:

This issue was mitigated as part of fix **F-2026-14846** by establishing an operational procedure for revoking contract access rather than implementing a code-level fix.

Mitigation Approach:

The `whitelistedFunctions` mapping is designated as the authoritative source of trust. The proper procedure for revoking access from a contract requires administrators to follow these steps **in order**:

1. Remove each whitelisted function associated with the contract using

`removeWhitelistedFunction`:

```
removeWhitelistedFunction(funcSignature1);
removeWhitelistedFunction(funcSignature2);
// ... repeat for all functions of the contract
```

2. Set the contract's access level to `NONE`:

```
setTrustedContract(contractAddress, ContractAccess.NONE);
```

[F-2026-14861](#) - Front-Running DoS on Batch Settlement via Rolling Hash Invalidation - Medium

Description:

The rolling hash mechanism used to ensure batch settlement integrity can be exploited by any user to indefinitely block settlements. Since each new deposit or withdrawal request modifies the rolling hash, an attacker can front-run settlement transactions to invalidate them, causing repeated reverts and preventing users from receiving shares or withdrawing assets.

The **OmniVaultManager** employs a rolling hash mechanism where each deposit and withdrawal request appends to the respective hash:

```
// In requestDeposit()
rollingDepositHash = keccak256(abi.encode(rollingDepositHash, requestId, _tokens, _amounts));

// In requestWithdrawal()
rollingWithdrawalHash = keccak256(abi.encode(rollingWithdrawalHash, requestId, _shares));
```

During settlement, the **SETTLER_ROLE** must provide data that reconstructs the exact rolling hash:

```
// In _bulkSettleDeposits()
require(depositHash == rollingDepositHash, "VM-DHMR-01");
// In _bulkSettleWithdrawals()
require(withdrawalHash == rollingWithdrawalHash, "VM-WHMR-01");
```

The vulnerability arises because any user can submit a new deposit or withdrawal request that changes the rolling hash. If this occurs after the settler has prepared their settlement transaction but before it confirms, the settlement reverts due to hash mismatch.

This can lead to:

- **Settlement blocked:** Deposits cannot be settled, preventing users from receiving vault shares.
- **Withdrawals blocked:** Withdrawals cannot be settled, preventing users from receiving their assets.
- **Temporary fund lock:** User deposits remain locked in the executor, and withdrawal shares remain locked in the manager.
- **Recovery delay:** The system can only recover when **MAX_PENDING_REQUESTS** reaches 1000 and no additional requests can be submitted, or via **unwindBatch** after **RECLAIM_DELAY** (24 hours), which refunds all pending requests without settlement.

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation:

- **Option 1: Per-Request Settlement Isolation:** Decouple individual requests from a shared rolling hash so that new requests do not invalidate previously submitted ones. Each request's data should be verifiable independently while maintaining fair ordering guarantees for share pricing. This allows the settler to process existing requests without being affected by concurrent new submissions.
- **Option 2: Request Window Separation:** Separate request submission and settlement into distinct time periods. New requests should be blocked during settlement processing to ensure the batch state remains stable. This prevents concurrent modifications to settlement data while preserving the integrity of batch ordering.

Resolution:

Fixed in `cf300d7`:

OmniVaultManager now separates request intake from settlement using batch finalization:

- `finalizeBatch` stores `rollingDepositHash` and `rollingWithdrawalHash` in `completedBatches[batchId]`, then starts a new batch via `_resetBatch`.
- `bulkSettleState` settles only the finalized previous batch (`currentBatchId - 1`) and verifies settlement data against stored batch hashes.
- New `requestDeposit` and `requestWithdrawal` calls update only current-batch rolling hashes, so finalized-batch settlement no longer reverts from concurrent request activity.

[F-2026-14863](#) - Missing Gas Tank Auto-Fill for Recipients in Bulk Transfers - Medium

Description:

The **PortfolioSub** contract implements an auto-fill mechanism (`autoFillPrivate`) designed to ensure users always have sufficient gas tokens (`ALOT`) in their Gas Tank to perform transactions on the Dexalot L1. According to the contract's documentation:

Users will always have some ALOT deposited to their gasTank if they start from the mainnet with any token. Hence it is not possible to have a portfolioSub holding without gas in the GasTank. In other words: if `assets[_trader][_symbol].available > 0` then `_trader.balance > 0`

However, this invariant is violated in `bulkTransferTokens` function, where token recipients do not receive the automatic gas refill that occurs in similar operations.

The single-token `transferToken` function correctly calls `autoFillPrivate` for the recipient:

```
function transferToken(
    address _to,
    bytes32 _symbol,
    uint256 _quantity
) external override whenNotPaused nonReentrant {
    require(tokenList.contains(_symbol), "P-ETNS-01");
    require(_to != msg.sender, "P-DOTS-01");
    require(tokenDetailsMap[_symbol].auctionMode == ITradePairs.AuctionMode.OF
F, "P-AUCT-01");
    transferToken(msg.sender, _to, _symbol, _quantity, 0, Tx.IXFERSENT, false,
address(0));
    autoFillPrivate(_to, _symbol, Tx.AUTOFILL);
}
```

However, the bulk transfer function omits this call:

```
function bulkTransferTokens(
    address _from,
    address _to,
    bytes32[] calldata _symbols,
    uint256[] calldata _quantities
) external whenNotPaused nonReentrant {
    require(_from == msg.sender || hasRole(TRUSTED_TRANSFER_ROLE, msg.sender),
"P-00WN-03");
```



```

require(_to != msg.sender, "P-DOTS-01");
require(_symbols.length == _quantities.length, "P-ARLM-01");
for (uint256 i = 0; i < _symbols.length; ) {
    bytes32 symbol = _symbols[i];
    uint256 quantity = _quantities[i];
    require(tokenList.contains(symbol), "P-ETNS-01");
    transferToken(_from, _to, symbol, quantity, 0, Tx.IXFERSENT, false, address(0));
    unchecked {
        i++;
    }
}
}

```

This may lead to the following consequences:

- Recipients cannot perform any transactions on Dexalot L1
- Users may believe their funds are lost or inaccessible
- Invariant violation: Breaks the documented guarantee: `assets[trader][symbol].available > 0` → `trader.balance > 0`
- Inconsistent behavior: Single transfers work correctly while bulk transfers leave users stranded

Assets:

- contracts/PortfolioSub.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Add `autoFillPrivate` to `bulkTransferTokens`:

```

autoFillPrivate(_to, _symbols[_symbols.length - 1], Tx.AUTOFILL);

```

Resolution:

Fixed in [bf20793](#):

The function now calls `autoFillPrivate(_to, _symbols[0], Tx.AUTOFILL)` after completing all transfers, ensuring the recipient's Gas Tank is refilled if needed.

```
autoFillPrivate(_to, _symbols[0], Tx.AUTOFILL);
```

[F-2026-14935](#) - Auction Mode Bypass in bulkTransferTokens Allows Transfers During Active Auctions - Medium

Description:

The `bulkTransferTokens` function does not validate auction mode before transferring tokens, unlike the public `transferToken` function. This allows tokens under auction restrictions to be transferred, potentially undermining auction integrity by enabling early distribution of auction tokens.

The public `transferToken` function includes an auction mode check:

```
require(tokenDetailsMap[_symbol].auctionMode == ITradePairs.AuctionMode.OFF,
"P-AUCT-01");
```

However, `bulkTransferTokens` calls the private `transferToken` function directly without this validation:

```
function bulkTransferTokens(
    address _from,
    address _to,
    bytes32[] calldata _symbols,
    uint256[] calldata _quantities
) external whenNotPaused nonReentrant {
    require(_from == msg.sender || hasRole(TRUSTED_TRANSFER_ROLE, msg.sender), "P-00WN-03");
    require(_to != msg.sender, "P-DOTS-01");
    require(_symbols.length == _quantities.length, "P-ARLM-01");
    for (uint256 i = 0; i < _symbols.length; ) {
        bytes32 symbol = _symbols[i];
        uint256 quantity = _quantities[i];
        require(tokenList.contains(symbol), "P-ETNS-01");
        // No auction mode check before calling private transferToken
        transferToken(_from, _to, symbol, quantity, 0, Tx.IXFERSENT, false, address(0));
        unchecked {
            i++;
        }
    }
}
```

This creates an inconsistency where single-token transfers respect auction restrictions while bulk transfers bypass them.

This can lead to:

- **Auction integrity violation:** Tokens under auction can be transferred before the auction concludes.

- **Early token distribution:** Auction tokens can be distributed to recipients who should not have access until the auction ends.
- **Inconsistent behavior:** Single and bulk transfer functions have different security guarantees for the same operation.

Assets:

- contracts/PortfolioSub.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: The auction mode check should be added to the `bulkTransferTokens` loop before calling the private `transferToken`:

```
require(tokenDetailsMap[symbol].auctionMode == ITradePairs.AuctionMode.OFF, "P-AUCT-01");
```

This ensures consistent auction mode enforcement regardless of whether single or bulk transfer is used.

Resolution: Fixed in `0e1526e`:

`bulkTransferTokens` now enforces the same auction-mode restriction as `transferToken` by adding:

```
require(tokenDetailsMap[symbol].auctionMode == ITradePairs.AuctionMode.OFF, "P-AUCT-01");
```

The bypass path described in the issue is removed, and transfer behavior is now consistent between single-transfer and bulk-transfer flows for auction-restricted tokens.

F-2026-14991 - depositTokenFromContract Cannot Pay Bridge Fees - Medium

Description:

The `depositTokenFromContract` function is used by trusted contracts to deposit tokens on behalf of users. Following the v2.6.3 upgrade, the deposit flow now requires native bridge fees to be paid via `msg.value`, which is forwarded to `portfolioBridge.sendXChainMessage{value: _nativeBridgeFee}(...)`.

However, `depositTokenFromContract` is not payable and does not forward any value when calling `depositToken`:

```
function depositTokenFromContract(address _from, bytes32 _symbol, uint256 _quantity) external override {
    require(trustedContracts[msg.sender], "P-A0TC-01");
    this.depositToken(_from, _symbol, _quantity, portfolioBridge.getDefaultBridgeProvider());
}
```

When `depositToken` is subsequently called:

1. `msg.value` will be `0`
2. The internal `deposit` function will call `portfolioBridge.sendXChainMessage{value: 0}(...)`
3. If the bridge provider requires a fee (`userPaysFee` returns `true`), the transaction will fail

This may lead to:

- **Breaking existing integrations:** Trusted contracts that previously used `depositTokenFromContract` will fail when bridge fees are enabled for the default bridge provider
- **Denial of Service for trusted contracts:** Trusted integrations cannot complete deposits
- **Inconsistent behavior:** Direct calls to `depositToken` can pay fees, but calls via `depositTokenFromContract` cannot

Assets:

- `contracts/PortfolioMain.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Medium

Recommendations

Remediation: Make `depositTokenFromContract` `payable` and forward the value to `depositToken`.

Resolution: Fixed in `fbe76ba`:

The issue has been resolved by making the following changes to the `depositTokenFromContract` function:

1. **Added `payable` modifier:** The function now accepts native currency via `msg.value`, allowing trusted contracts to include bridge fees with their deposit calls.
2. **Forward `msg.value` to `depositToken`:** The internal call to `depositToken` now includes `{value: msg.value}`, ensuring that any native bridge fees sent by the trusted contract are properly forwarded through the deposit flow to `portfolioBridge.sendXChainMessage`.

```
function depositTokenFromContract(address _from, bytes32 _symbol, uint256 _quantity) external payable override {
    require(trustedContracts[msg.sender], "P-A0TC-01");
    this.depositToken{value: msg.value}(_from, _symbol, _quantity, portfolioBridge.getDefaultBridgeProvider());
}
```

[F-2026-14835](#) - Denial of Service via Global Pending Request Limit Exhaustion - Low

Description:

The **OmniVaultManager** contract implements a global `pendingRequestCount` counter with a hard cap of `MAX_PENDING_REQUESTS = 1000`. This counter is shared across all vaults and is only reset when a batch is settled or unwound. An attacker can exhaust this limit by creating numerous minimal deposit requests, temporarily blocking all deposit and withdrawal operations for all vaults in the system.

The pending request counter is global and incremented for both deposits and withdrawals:

```
uint256 public constant MAX_PENDING_REQUESTS = 1000;
// ...
uint256 public pendingRequestCount;
```

Both `requestDeposit` and `requestWithdrawal` check and increment this counter:

```
function requestDeposit(
    uint256 _vaultId,
    uint16[] calldata _tokens,
    uint256[] calldata _amounts
) external nonReentrant whenNotPaused returns (bytes32 requestId) {
    // ...
    require(pendingRequestCount < MAX_PENDING_REQUESTS, "OV-MPRC-01");
    _depositTokens(_tokens, _amounts, vaultDetails[_vaultId].tokens, executor);
    ;
    requestId = _generateRequestId(_vaultId, msg.sender, userNonce[msg.sender]
    ++);
    pendingRequestCount++;
    // ...
}
```

The counter is only reset in `_resetBatch`, which is called after `bulkSettleState` or `unwindBatch`:

```
function _resetBatch() internal {
    rollingDepositHash = 0;
    rollingWithdrawalHash = 0;
    pendingRequestCount = 0;
    batchStartTime = block.timestamp;
    currentBatchId++;
}
```

Attack Scenario

1. Attacker identifies the minimum deposit amount (`minPerDeposit`) for any supported token
2. Attacker creates 1000 deposit requests with minimum amounts
3. All vaults are blocked from accepting new deposits or withdrawals
4. Legitimate users cannot interact with ANY vault until:
 1. Settler role calls `bulkSettleState`, OR
 2. 24 hours pass and anyone calls `unwindBatch`
5. Attacker's tokens are refunded via settle/unwind (no permanent loss)

Status: Mitigated

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider implementing per-vault and per-user request limits.

Resolution: Mitigated in `e7f9964`:

The original global cap mechanism remains in place, while additional throttling controls have been introduced. Current implementation sets `MAX_PENDING_REQUESTS = 500` and enforces per-vault and per-user limits through `MAX_VAULT_PENDING_REQUESTS = 50` and `MAX_USER_PENDING_REQUESTS = 5`, applied in `_verifyAndIncrementRequestLimits` for both deposit and withdrawal requests.

```
uint256 public constant MAX_PENDING_REQUESTS = 500;
uint256 public constant MAX_VAULT_PENDING_REQUESTS = 50;
uint256 public constant MAX_USER_PENDING_REQUESTS = 5;
```

This materially reduces request-spam impact from a single actor, but the shared global `pendingRequestCount` gate is still present and can still halt intake when fully consumed.

[F-2026-14837](#) - Vault Status Management Allows Setting NONE/DEPRECATED Status, Locking User Funds - Low

Description:

The **OmniVaultManager** contract's vault status management system allows an admin to set a vault's status to **NONE** or **DEPRECATED** through **updateVaultDetails**, which permanently locks user funds with no recovery mechanism.

```
enum VaultStatus {  
    NONE,  
    ACTIVE,  
    PAUSED,  
    DEPRECATED  
}
```

State Transition Functions

- **pauseVault** - Transitions **ACTIVE** → **PAUSED** only:

```
function pauseVault(uint256 _vaultId) external onlyRole(DEFAULT_ADMIN_ROLE) {  
    VaultDetails storage vaultDetail = vaultDetails[_vaultId];  
    require(vaultDetail.status != VaultStatus.NONE, "OV-PVDA-01");  
    vaultDetail.status = VaultStatus.PAUSED;  
}
```

- **unpauseVault** - Transitions **PAUSED** → **ACTIVE** only:

```
function unpauseVault(uint256 _vaultId) external onlyRole(DEFAULT_ADMIN_ROLE)  
{  
    VaultDetails storage vaultDetail = vaultDetails[_vaultId];  
    require(vaultDetail.status == VaultStatus.PAUSED, "VM-VINP-01");  
    vaultDetail.status = VaultStatus.ACTIVE;  
}
```

- **updateVaultDetails** - Can set any status

```
function updateVaultDetails(  
    uint256 _vaultId,  
    VaultDetails calldata _vaultDetails  
) external onlyRole(DEFAULT_ADMIN_ROLE) {  
    VaultDetails storage vaultDetail = vaultDetails[_vaultId];  
    require(vaultDetail.status != VaultStatus.PAUSED, "OV-UVDA-01"); // Blocks  
    if PAUSED
```

```
// ...
vaultDetails[_vaultId] = _vaultDetails; // Overwrites entire struct including status!
}
```

- Request functions status checks:

```
// requestDeposit - ONLY ACTIVE
require(status == VaultStatus.ACTIVE, "OV-RDPR-01");
// requestWithdrawal - ACTIVE or PAUSED (not NONE, not DEPRECATED!)
require(status == VaultStatus.ACTIVE || status == VaultStatus.PAUSED, "OV-RDPR-01");
```

NONE or **DEPRECATED** Status Creates Permanent Fund Lock:

1. Admin pauses vault (**ACTIVE** → **PAUSED**)
2. Admin calls `updateVaultDetails` with `status = NONE` or `status = DEPRECATED`
3. Now both deposits AND withdrawals become impossible:

- `requestDeposit`: fails because `status != ACTIVE`
- `requestWithdrawal`: fails because `status != ACTIVE && status != PAUSED`

No standard recovery path exists within the contract:

- `pauseVault`: requires `status == ACTIVE` (fails)
- `unpauseVault`: requires `status == PAUSED` (fails)
- `updateVaultDetails`: requires `status == PAUSED` (fails)

While the contract is upgradeable (allowing eventual recovery through a contract upgrade), this represents a design flaw that:

- Should not require an emergency upgrade to resolve
- Leaves user funds inaccessible until an upgrade is deployed
- Introduces unnecessary operational risk and potential delays in fund recovery

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity:

Low

Recommendations

Remediation:

- Add dedicated status transition functions.
- Allow withdrawals for **DEPRECATED** vaults.

Resolution:

Fixed in **b7af933**:

Current **updateVaultDetails** no longer overwrites vault status and is restricted to paused vaults via **require(vault.status == VaultStatus.PAUSED, "VM-VSNP-01")**. Status changes are handled through dedicated functions (**pauseVault**, **unpauseVault**, **deprecateVault**).

```
function deprecateVault(uint256 _vaultId) external onlyRole(DEFAULT_ADMIN_ROLE) {
    VaultDetails storage vaultDetail = vaultDetails[_vaultId];
    require(vaultDetail.status != VaultStatus.NONE, "VM-VNA-01");
    require(_getPendingVaultRequests(_vaultId) == 0, "VM-DVPR-01");
    vaultDetail.status = VaultStatus.DEPRECATED;
}
```

Withdrawal requests are not blocked for deprecated vaults, since **requestWithdrawal** rejects only Vault **Status.NONE**. The permanent lock path described in the issue is not present in the current implementation.

```
require(status != VaultStatus.NONE, "VM-VSAP-01");
```

[F-2026-14838](#) - Unrestricted Share Minting by Settler Role Enables Value Extraction from Depositors - Low

Description:

The `SETTLER_ROLE` can specify arbitrary `depositShares` values when settling deposit requests through `bulkSettleState`, without any on-chain validation mechanism to verify that the minted shares correspond to the fair value of deposited tokens. A malicious or compromised settler can exploit this to systematically extract value from depositors by minting fewer shares than deserved, or dilute existing share holders by minting excessive shares.

The `_bulkSettleDeposits` function processes deposit fulfillments by iterating through the provided `DepositFulfillment` array and minting shares directly based on the `depositShares` value supplied by the caller with `SETTLER_ROLE`. The rolling hash mechanism, which is intended to ensure integrity between deposit requests and settlements, only validates the `depositRequestId`, `tokenIds`, and `amounts` fields:

```
depositHash = keccak256(abi.encode(depositHash, requestId, _deposits[i].tokenIds, _deposits[i].amounts));
```

The `depositShares` value is notably absent from the hash calculation. This means the settler can provide any arbitrary share amount, and as long as the other fields match the original deposit request, the settlement will succeed. The share minting occurs unconditionally when `depositShares` is non-zero:

```
if (_deposits[i].depositShares != 0) {  
    IOmniVaultShare(vaultDetails[vaultId].shareToken).mint(vaultId, user, _deposits[i].depositShares);  
    continue;  
}
```

There is no on-chain reference point for:

- The current share price or Net Asset Value (NAV)
- Expected share amount based on deposited token values
- Price bounds or acceptable deviation thresholds
- Historical share price for consistency checks

The `TransferRequest` struct stored during `requestDeposit` only contains status, timestamp, and shares (initialized to 0 for deposits), providing no data against which to validate the settlement parameters.

The potential impact of this vulnerability is:

- **Value extraction from depositors:** A compromised settler can mint fewer shares than the fair value entitles, effectively stealing a portion of each deposit.
- **Dilution of existing holders:** Excessive share minting for new deposits dilutes the ownership percentage of all existing vault participants. This transfers value from existing holders to new depositors (or potentially to the attacker if they coordinate with the new depositor).
- **Systematic exploitation potential:** Since settlements occur in batches, a single malicious settlement call can affect multiple users simultaneously, amplifying the total extractable value.
- **Difficulty of detection:** Users have no on-chain mechanism to verify they received fair share amounts, relying entirely on off-chain transparency and trust in the settlement service.

Assets:

- contracts/vaults/OmniVaultManager.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

To mitigate the risks associated with unvalidated `depositShares` values during deposit settlement, the following safeguards are recommended:

- **Clear Documentation of Trust Model:**
Clearly disclose in protocol documentation that depositors rely on the integrity of the `SETTLER_ROLE` for fair share issuance. It should be made explicit that the protocol does not enforce on-chain validation of `depositShares`, and that this role carries the authority to determine share allocation at settlement time.
- **Deviation Thresholds Based on Vault NAV:**
Introduce a configurable deviation threshold (e.g., $\pm 1\text{--}20\%$) comparing the implied share price (deposit value / depositShares) with the vault's current NAV per share at settlement. If the deviation exceeds this

threshold, the settlement should be flagged or require additional approvals.

- **Settlement Transparency Events:**

Emit detailed events for each deposit settlement including:

- Vault NAV and total shares at settlement time
- Deposited token values
- `depositShares` minted
- Implied share price for the transaction

This allows off-chain validators, auditors, and users to independently verify fairness and detect anomalies post-settlement.

Resolution:

Fixed in `7d3aa10`:

`depositShares` is no longer an external settlement input. `DepositFulfillment` was changed from `depositShares` to `process`, and `_bulkSettleDeposits` now computes minted shares on-chain from deposited token value, vault USD state, and total shares.

Arbitrary settler-controlled share mint amounts described in the issue are removed from the settlement interface.

[F-2026-14840](#) - Unrestricted Withdrawal Amounts by Settler Role Enables Value Extraction from Withdrawing Users - Low

Description:

The `SETTLER_ROLE` can specify arbitrary symbols and amounts arrays when settling withdrawal requests through `bulkSettleState`, without any on-chain validation that the dispatched assets correspond to the user's proportional share of vault holdings. A malicious or compromised settler can exploit this to systematically underpay withdrawing users, retaining excess vault assets.

The `_bulkSettleWithdrawals` function processes withdrawal fulfillments by iterating through the provided `WithdrawalFulfillment` array and dispatching assets based entirely on the symbols and amounts values supplied by the caller with `SETTLER_ROLE`. The rolling hash mechanism only validates the `withdrawalRequestId` and `wRequest.shares`:

```
withdrawalHash = keccak256(abi.encode(withdrawalHash, item.withdrawalRequestId, wRequest.shares));
```

The amounts arrays is notably absent from the hash calculation. This means the settler can provide any arbitrary quantities, and as long as the request ID and shares match the original withdrawal request, the settlement will succeed. The asset dispatch occurs without validation of fair value:

```
if (_withdrawals[i].symbols.length > 0) {
    shareToken.burn(vaultId, uint256(wRequest.shares));
    IOmniVaultExecutorSub(vaultDetail.executor).dispatchAssets(
        user,
        _withdrawals[i].symbols,
        _withdrawals[i].amounts
    );
}
```

There is no on-chain reference point for:

- Current vault NAV or total asset values
- Proportional share of each asset type the user is entitled to
- Minimum acceptable amounts based on share percentage
- Price validation against oracles or committed values

The only validation performed is array length matching between symbols and amounts, which does not constrain the actual values.

The potential impact of this vulnerability includes:

- **Value extraction from withdrawing users:** A compromised settler can dispatch fewer assets than the user's shares entitle, effectively stealing a portion of each withdrawal.
- **Selective token manipulation:** The settler controls which tokens are dispatched, potentially forcing users to receive less desirable or more volatile assets while retaining higher-value tokens in the vault.
- **Cumulative extraction:** Since withdrawals are processed in batches, systematic underpayment across many users compounds the extractable value over time.

Assets:

- contracts/vaults/OmniVaultManager.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

To mitigate the risks introduced by the design's reliance on **SETTLER_ROLE**, the following measures are recommended:

- **Clear Documentation of Trust Assumptions:**
Explicitly state in protocol documentation that vault operations rely on the integrity of the **SETTLER_ROLE**, and that withdrawals are fulfilled based on discretionary asset allocation at settlement time. Users should be informed of the inherent trust trade-offs.
- **Deviation Thresholds with Failsafes:**
Implement a configurable threshold (e.g., $\pm 1\text{--}20\%$) for acceptable deviation between expected and actual withdrawal value based on current NAV per share. If a settlement exceeds this range, it can be flagged, paused, or require multisig approval. For example, if 1,000 shares correspond to \$10,000 in assets, a deviation over \$2000 (20%) would trigger a failsafe.
- **Detailed Transparency Events:**
Emit comprehensive events during settlement that include:
 - Calculated NAV at the time of settlement

- Total vault asset composition
- User share percentage and asset breakdown
- Dispatched tokens and amounts

These logs enable off-chain auditing and user-level verification of fair treatment.

Resolution:

Fixed in `7d3aa10`:

The described scenario no longer exists. `WithdrawalFulfillment` no longer accepts caller-supplied symbols and amounts; settlement input is reduced to `withdrawalRequestId` and process.

`_bulkSettleWithdrawals` derives token IDs from vault configuration and computes dispatch amounts on-chain from vault balances and share proportion, then dispatches those computed values. Arbitrary settler-provided token/amount injection for withdrawal settlement is removed.

[F-2026-14841](#) - Missing Token Validation in Withdrawal Settlement Allows Arbitrary Asset Dispatch - Low

Description:

The `_bulkSettleWithdrawals` function lacks explicit validation that the provided token symbols belong to the vault's configured token list, unlike the deposit flow which enforces this check via `_tokenExistsInVault`. This asymmetry allows the settler to specify arbitrary tokens or duplicate the same token multiple times within a single withdrawal fulfillment.

During deposit requests, the `_depositTokens` function explicitly validates that each deposited token exists in the vault's configured token list:

```
function _depositTokens(..., uint16[] memory _vaultTokens, ...) internal {
    // ...
    for (uint256 i = 0; i < len; i++) {
        // ...
        require(_tokenExistsInVault(tokenId, _vaultTokens), "VM-TIIV-01");
        // ...
    }
    portfolio.bulkTransferTokens(msg.sender, _executor, symbols, _amounts);
}
```

The rolling hash mechanism further ensures that settlement must match the exact tokens deposited.

However, in `_bulkSettleWithdrawals`, no equivalent validation exists:

```
function _bulkSettleWithdrawals(WithdrawalFulfillment[] calldata _withdrawals)
internal {
    // ...
    // Rolling hash only captures requestId + shares
    withdrawalHash = keccak256(abi.encode(withdrawalHash, item.withdrawalRequestId, wRequest.shares));
    // ...
    // No validation that symbols are in vault's token list
    if (_withdrawals[i].symbols.length > 0) {
        IOmniVaultExecutorSub(vaultDetail.executor).dispatchAssets(
            user,
            _withdrawals[i].symbols, // Settler provides any symbols
            _withdrawals[i].amounts
        );
    }
    // ...
}
```

The settler can provide:

- Tokens not in the vault's configured token list
- Duplicate tokens within the same withdrawal (e.g., the same token appearing multiple times in the symbols array)

Current protection relies solely on the practical constraint that the executor's **PortfolioSub** balance should only contain vault tokens. If the executor acquires tokens outside the vault's list through any mechanism, the settler could dispatch them to users or themselves.

This can lead to:

- **Inconsistent security model:** Deposits enforce explicit token validation while withdrawals rely on implicit balance constraints, creating an asymmetric trust model.
- **Duplicate token exploitation:** A settler can include the same token multiple times in a withdrawal fulfillment, potentially manipulating the distribution of assets.
- **Defense-in-depth violation:** The absence of explicit validation means any unexpected executor balance (from airdrops, erroneous transfers, or other sources) could be extracted without on-chain constraints.

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- Explicit token validation should be added to `_bulkSettleWithdrawals` to match the deposit flow's security guarantees. Each symbol provided in the withdrawal fulfillment should be validated to ensure it belongs to the specific vault's configured token list, not just the global **PortfolioSub** token registry.

- Duplicate symbols within the same withdrawal request should be rejected. A dedicated helper function can encapsulate this validation logic, ensuring consistent security guarantees between deposit and withdrawal flows.

Resolution:

Fixed in `7d3aa10`:

Withdrawal settlement no longer accepts settler-supplied token symbol and amount arrays. `WithdrawalFulfillment` was changed to process control only, and `_bulkSettleWithdrawals` now derives token IDs from vault configuration and computes dispatch amounts from vault state.

This removes arbitrary-token and duplicate-symbol injection through withdrawal fulfillment inputs and aligns withdrawal token selection with vault-defined assets.

[F-2026-14845](#) - fallback Ignores ContractAccess Type, Allowing Function Execution on Contracts With Restricted Access Levels - Low

Description:

The **OmniVaultExecutor** defines granular access levels for trusted contracts via the `ContractAccess` `enum (NONE, TRUSTED, NATIVE, ERC20, NATIVE_AND_ERC20)`. However, the `fallback` function ignores these access levels when executing whitelisted functions.

The `ContractAccess` enum defines specific permission levels:

```
enum ContractAccess {  
    NONE,           // Not trusted  
    TRUSTED,        // Only for whitelisted function calls (implied)  
    NATIVE,          // Only for sending native currency  
    ERC20,           // Only for ERC20 approvals  
    NATIVE_AND_ERC20 // Both native and ERC20 operations  
}
```

These access levels are enforced in `sendNative` and `approveToken`:

```
function sendNative(address payable _to, uint256 _amount) external onlyRole(0  
MNITRADER_ROLE) {  
    ContractAccess access = trustedContracts[_to];  
    require(access == ContractAccess.NATIVE || access == ContractAccess.NATIVE  
_AND_ERC20, "0T-IVCA-01");  
    // ...  
}
```

```
function approveToken(address _token, address _spender, uint256 _amount) exte  
rnal onlyRole(MNITRADER_ROLE) {  
    ContractAccess access = trustedContracts[_spender];  
    require(access == ContractAccess.ERC20 || access == ContractAccess.NATIVE_  
AND_ERC20, "0T-IVCA-01");  
    // ...  
}
```

But `fallback` does not check access level:

```
fallback() external payable onlyRole(MNITRADER_ROLE) {  
    require(msg.data.length >= 4, "0T-FSNV-01");  
    address impl = whitelistedFunctions[msg.sig];  
    require(impl != address(0), "0T-FSNW-01");
```

```
(bool success, ) = impl.call{value: msg.value}(msg.data);
_returnData(success);
}
```

This may lead to:

- **Access Control Bypass:** Contracts meant only for native/ERC20 operations can have arbitrary functions called
- **Principle of Least Privilege Violation:** Granular access levels become meaningless for function calls
- **Confusion:** Admin may think `NATIVE` access limits a contract to `sendNative` only

Assets:

- contracts/vaults/OmniVaultExecutor.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate: 5/5

Likelihood Rate: 1/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Check access type in `fallback`.
- Document that `NATIVE` / `ERC20` also allow function calls: If this is intended behavior, update documentation to clarify that all non-`NONE` access levels allow whitelisted function execution.

Resolution:

Mitigated (no on-chain code change)

The comments explicitly document intended semantics that non-`NONE` access tiers may execute whitelisted functions, reducing operator misunderstanding. Runtime enforcement in `fallback` still does not gate execution by `ContractAccess`, so no hard access-control restriction was added at code-execution level.

```
enum ContractAccess {  
    NONE, // Not trusted, cannot call any functions or send/receive funds  
    TRUSTED, // Trusted contract, can call whitelisted functions that don't s  
end/receive funds  
    NATIVE, // Trusted contract, can call whitelisted functions that send/rec  
eive native currency  
    ERC20, // Trusted contract, can call whitelisted functions that send/rece  
ive ERC20 tokens  
    NATIVE_AND_ERC20 // Trusted contract, can call whitelisted functions that  
send/receive both native currency and ERC20 tokens  
}
```

[F-2026-14846](#) - No Mechanism to Remove Whitelisted Functions in OmniVaultExecutor - Low

Description:

The **OmniVaultExecutor** contract allows admin to whitelist functions by mapping function selectors to trusted contract addresses. However, there is no mechanism to remove or disable a whitelisted function once it has been added. The only way to "change" a whitelisted function is to overwrite it with a different trusted contract, but setting it to `address(0)` is not possible due to the trust validation check.

The `_setWhitelistedFunction` function requires the target contract to be trusted:

```
function _setWhitelistedFunction(bytes4 _funcSignature, address _contract) internal {
    require(trustedContracts[_contract] != ContractAccess.NONE, "OT-IVTC-01");
    // Must be trusted!
    whitelistedFunctions[_funcSignature] = _contract;
}
```

To remove a whitelisted function, admin would need to set it to `address(0)`:

```
// Attempting to remove:
executor.setWhitelistedFunction(selector, address(0));
// This fails because:
// trustedContracts[address(0)] == ContractAccess.NONE
// require fails with "OT-IVTC-01"
```

This may block the following scenarios:

- Remove dangerous function
- Temporarily disable function
- Emergency response
- Clean up unused selectors

Assets:

- `contracts/vaults/OmniVaultExecutor.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:

4/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Add explicit remove function:

```
function removeWhitelistedFunction(bytes4 _funcSignature) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(whitelistedFunctions[_funcSignature] != address(0), "OT-FNNW-01");
    // Function not whitelisted

    address oldContract = whitelistedFunctions[_funcSignature];
    delete whitelistedFunctions[_funcSignature];

    emit WhitelistedFunctionRemoved(_funcSignature, oldContract);
}
```

Resolution: Fixed in [9bead9b](#):

An explicit removal path exists in **OmniVaultExecutor** through `removeWhitelistedFunction(bytes4)`, restricted to `DEFAULT_ADMIN_ROLE`. The function validates selector presence, deletes the mapping entry, and emits `WhitelistedFunctionUpdate(_funcSignature, address(0))`.

```
function removeWhitelistedFunction(bytes4 _funcSignature) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(whitelistedFunctions[_funcSignature] != address(0), "VE-FSNW-01");
    ;
    delete whitelistedFunctions[_funcSignature];
}
```

F-2026-14850 - First Batch Vulnerable to Immediate Unwind - Low

Description:

The `batchStartTime` state variable is not initialized during contract deployment or in the `initialize` function, defaulting to 0. The `unwindBatch` function allows anyone to unwind pending requests after `RECLAIM_DELAY` (24 hours) has passed since `batchStartTime`.

```
uint256 public batchStartTime;

function unwindBatch(
    DepositFulfillment[] calldata _deposits,
    WithdrawalFulfillment[] calldata _withdrawals
) external nonReentrant {
    // Since batchStartTime = 0 at deployment, the condition becomes:
    require(block.timestamp >= 0 + 24 hours);
    // Equivalent to: block.timestamp >= 86400
    ...
}
```

The current Unix timestamp is approximately 1.7 billion seconds (since January 1, 1970), which is always greater than 86,400 seconds (24 hours). This means the time check always passes immediately after deployment.

Assets:

- contracts/vaults/OmniVaultManager.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Initialize `batchStartTime` to the current timestamp during contract initialization:

Resolution:

Fixed in [45ab5a5](#):

`batchStartTime` is initialized in `initialize` with `batchStartTime = block.timestamp`, preventing immediate eligibility of `unwindBatch` after deployment. Batch timing is subsequently maintained through `_resetBatch`, which updates `batchStartTime` to the current block timestamp at each batch transition.

```
batchStartTime = block.timestamp;
```

[F-2026-14869](#) - TRUSTED_TRANSFER_ROLE Allows Arbitrary Fund Transfers - Low

Description: The **PortfolioSub** contract defines a **TRUSTED_TRANSFER_ROLE** that grants privileged addresses the ability to transfer tokens from any user's portfolio to any destination address:

```
bytes32 public constant TRUSTED_TRANSFER_ROLE = keccak256("TRUSTED_TRANSFER_ROLE");
```

This role is used in the **bulkTransferTokens** function:

```
function bulkTransferTokens(
    address _from,
    address _to,
    bytes32[] calldata _symbols,
    uint256[] calldata _quantities
) external whenNotPaused nonReentrant {
    require(_from == msg.sender || hasRole(TRUSTED_TRANSFER_ROLE, msg.sender), "P-00WN-03");
    require(_to != msg.sender, "P-DOTS-01");
    require(_symbols.length == _quantities.length, "P-ARLM-01");
    for (uint256 i = 0; i < _symbols.length; ) {
        bytes32 symbol = _symbols[i];
        uint256 quantity = _quantities[i];
        require(tokenList.contains(symbol), "P-ETNS-01");
        transferToken(_from, _to, symbol, quantity, 0, Tx.IXFERSENT, false, address(0));
        unchecked {
            i++;
        }
    }
}
```

This leads to the centralization risk:

- Total Value at Risk (TVaR) = 100% of all user deposits in **PortfolioSub**
- Single point of failure for all user funds

Assets:

- contracts/PortfolioSub.sol
- [<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate:	5/5
Likelihood Rate:	1/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Consider removing the `TRUSTED_TRANSFER_ROLE` functionality or clearly disclose this trust assumption in user-facing documentation and implement the following role assignment policy:

- Only assign to audited, immutable contracts
- Never assign to EOAs
- Document all addresses holding this role

Resolution: **Mitigated (no on-chain code change)**

`TRUSTED_TRANSFER_ROLE` remains enabled, while assignment policy is explicitly documented to restrict usage to the **OmniVaultManager** contract only. This reduces risk through governance and transparency controls, but the privileged transfer mechanism remains in place and centralization risk is not fully eliminated.

F-2026-14874 - Strict Payload Length Validation May Reject In-Flight Messages During Upgrade - Low

Description:

The `unpackXferMessage` function introduces strict payload length validation that differs from the previous implementation's behavior. This change may cause in-flight cross-chain messages sent before an upgrade to be rejected after the upgrade is applied, potentially resulting in stuck or lost funds.

The updated function adds explicit payload length checks:

```
if (xfer.transaction == IPortfolio.Tx.CCTRADE) {
    /// Check for exactly 148 bytes (128 + 20)
    require(_payload.length == 148, "PB-UM-02");
    senderAddress = address(bytes20(_payload[128:148]));
} else {
    require(_payload.length == 128, "PB-UM-03");
}
```

The initial version used `abi.decode(_payload, (bytes32[4]))` which would accept payloads of any length greater than or equal to 128 bytes, ignoring any trailing data. The new version uses slice notation `_payload[:128]` for decoding and enforces exact length requirements.

This behavioral change creates a compatibility gap: messages constructed and sent under the old format expectations may include additional data or have slightly different lengths that were previously tolerated but will now be rejected.

This can lead to:

- **In-flight message rejection:** Cross-chain messages that are in transit during the upgrade window will fail validation if their payload length does not exactly match the new requirements.
- **Potential fund loss:** If the rejected messages contain token transfer instructions, the associated funds may become stuck or require manual intervention to recover.
- **Cross-chain desynchronization:** Messages from older contract versions on remote chains that have not yet been upgraded will be rejected, causing asymmetric behavior across the deployment.

Assets:

- `contracts/PortfolioBridgeMain.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Before deploying this upgrade, all in-flight messages should be confirmed as processed and settled. A monitoring period can be implemented to ensure no pending cross-chain messages exist in bridge queues. If immediate upgrade is necessary, consider temporarily accepting both old and new payload formats with appropriate validation logic, then removing legacy support after a safe transition period.

Resolution: Mitigated:

Strict payload length validation remains in `unpackXferMessage`, and dual-format on-chain compatibility is not implemented. Risk of in-flight rejection is reduced through operational controls: `xChain` quote issuance pause, expiry-window wait, pending-message verification, and coordinated chain-by-chain upgrade sequencing before re-enabling quote generation.

Client comment:

Correct to mitigate this CCTrade messages are only generated by xChainSwaps which are generated and signed by an off-chain component. This component sets a 30 second expiry time for any quote it generates. Therefore for the upgrade we will pause giving out any xChainSwap quotes, after 30 seconds we will wait for all in flight messages to complete. Once we've verified there are no pending messages we will upgrade the contract chain by chain, once all chains are complete we can reenale our off-chain component. With this process we won't need a period of both payloads being supported

[F-2026-14875](#) - Unvalidated RFQ Address from Cross-Chain Payload Enables Potential Fund Misdirection - Low

Description:

The `processPayload` function extracts an `rfqAddress` from cross-chain payloads for `CCTRADE` transactions and directly calls `processXferPayload` on this address without on-chain validation that it is a legitimate, trusted **DexalotRFQ** contract. This design relies on an implicit assumption of deterministic `CREATE3` deployment across all chains, which if violated could result in stuck or lost funds.

When a `CCTRADE` is initiated on the source chain, `packXferMessage` appends the caller's address to the message payload:

```
if (_xfer.transaction == IPortfolio.Tx.CCTRADE) {
    bytes20 slot4 = bytes20(msg.sender);
    message = bytes.concat(slot0, slot1, slot2, slot3, slot4);
}
```

On the destination chain, `processPayload` extracts this address and routes the `CCTRADE` message without validation:

```
xfer.transaction == IPortfolio.Tx.CCTRADE
    ? IMainnetRFQ(payable(rfqAddress)).processXferPayload(xfer)
    : portfolio.processXferPayload(xfer);
```

The design relies on an implicit assumption that **DexalotRFQ** contracts are deployed at identical addresses across all chains using `CREATE3` deterministic deployment. There is no on-chain enforcement that:

- The `rfqAddress` corresponds to a legitimate **DexalotRFQ** contract on the destination chain
- The contract at `rfqAddress` has been properly initialized with correct role configurations
- The deployment order across chains was executed correctly

This flow affects `CCTRADE` transactions between mainnet chains. `DEPOSIT` and `WITHDRAW` transactions are unaffected as they route to `portfolio.processXferPayload`. The **PortfolioBridgeSub** explicitly blocks `CCTRADE` transactions.

If **DexalotRFQ** addresses differ across chains due to deployment order issues or non-deterministic deployment, the call on the destination chain would revert or call an incorrect address. User funds would remain locked on the source chain.

Assets:

- contracts/PortfolioBridgeMain.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status:

Mitigated

Classification

Impact Rate:

4/5

Likelihood Rate:

1/5

Exploitability:

Independent

Complexity:

Simple

Severity:

Low

Recommendations

Remediation:

- **Option 1: Trusted RFQ Registry (Recommended)**

- Implement an on-chain registry of trusted RFQ contract addresses that must be explicitly approved by admin before receiving **CCTRADE** messages. The **processPayload** function should validate that **rfqAddress** exists in this registry before routing the call. This provides defense-in-depth by requiring explicit admin approval on each chain, preventing both deployment order issues and front-running attacks.

- **Option 2: Document Deployment Assumptions**

- If **CREATE3** deterministic deployment is a hard requirement and on-chain validation is not desired, the critical assumption should be formally documented. Deployment runbooks should enforce correct deployment order when adding new chains, and verification scripts should validate RFQ addresses match across all chains before enabling cross-chain trading.

Resolution:

Mitigated in **ba7b1dc**:

Deployment-address assumptions for **CCTRADE** RFQ routing were explicitly documented in **PortfolioBridgeMain**, formalizing the CREATE3-based trust and routing model.

F-2026-14878 - Removal of Pause Functionality Eliminates Emergency Response Capability - Low

Description:

The **DexalotRFQ** contract removes all pause functionality that existed in **MainnetRFQ**, eliminating the ability to halt operations during security incidents. The previous implementation inherited **PausableUpgradeable** with `pause` / `unpause` functions callable by `DEFAULT_ADMIN_ROLE`, while the new contract has no mechanism to stop operations in emergencies.

The previous **MainnetRFQ** implementation included pause controls:

```
function pause() external override onlyRole(DEFAULT_ADMIN_ROLE) {
    _pause();
}

function unpause() external override onlyRole(DEFAULT_ADMIN_ROLE) {
    _unpause();
}
```

These functions allowed administrators to immediately halt all swap operations when security concerns arose. The new **DexalotRFQ** contract removes this capability entirely as part of the architectural decision to make the contract non-upgradeable.

Without pause functionality, once the contract is deployed there is no way to stop swap operations regardless of circumstances.

As a result:

- Swap operations cannot be stopped during active security incidents, allowing exploitation to continue until funds are depleted or the issue resolves naturally.
- In-flight cross-chain messages cannot be prevented, even if the source chain operation is known to be malicious.

Assets:

- contracts/DexalotRFQ.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: A minimal pause mechanism should be implemented that halts core swap functions including `simpleSwap`, `partialSwap`, and `xChainSwap`. This can be achieved through a simple boolean state variable and a `whenNotPaused` modifier, with `pause` and `unpause` functions restricted to `DEFAULT_ADMIN_ROLE`. This restores emergency response capability while maintaining the non-upgradeable contract design.

Resolution: Mitigated:

On-chain emergency pause controls remain absent in **DexalotRFQ**. Risk reduction is provided through operational control of quote signing and distribution, which can halt new swap execution paths without contract-level pause state transitions.

Pausing off-chain quote generation does not invalidate already signed quotes. Any signature already issued remains executable on-chain until expiry and other on-chain checks fail.

Mitigation effect is limited to preventing creation of new valid quotes, not revoking previously distributed signatures.

Client comment:

Intentional design decision to save gas. All swaps are generated by an off-chain component which signs quote messages so swaps can be executed. If we ever need to pause a given contract or all RFQs we simply manage this by pausing quote generation from our off-chain component this essentially provides the same restrictions as pausing. Also we're able to complete this much quicker than making a call on chain as we would need to call multiple RFQs on multiple mainnets.

F-2026-14879 - Immutable swapSigner Prevents Key Rotation and Creates Irrecoverable Compromise Risk - Low

Description:

The `swapSigner` address is declared as `immutable` in **DexalotRFQ**, meaning it can only be set during contract construction and cannot be changed afterward. If this signing key is compromised, attackers can generate unlimited fraudulent quotes at favorable rates, and the only recovery path requires deploying an entirely new contract.

The **DexalotRFQ** contract declares the swap signer as `immutable`:

```
address public immutable swapSigner;
```

The previous **MainnetRFQ** implementation allowed key rotation by the admin:

```
function setSwapSigner(address _swapSigner) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_swapSigner != address(0), "RF-SAZ-01");
    swapSigner = _swapSigner;
    emit SwapSignerUpdated(_swapSigner);
}
```

The swap signer is a critical security component that authorizes all swap quotes. This key is typically held by an off-chain service that generates quotes for users. Making it `immutable` removes the ability to:

- Rotate the key on a regular security schedule
- Respond to suspected key compromise
- Migrate to new key management infrastructure
- Recover from operational security incidents

A compromised signer key allows attackers to sign quotes at arbitrary rates, extracting value from the contract until drained or discovered.

Assets:

- contracts/DexalotRFQ.sol
- [<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate:

5/5

Likelihood Rate:	1/5
Exploitability:	Semi-Dependent
Complexity:	Medium
Severity:	Low

Recommendations

Remediation: A time-delayed signer rotation mechanism should be considered to balance security with operational flexibility. This would involve a two-step process: proposing a new signer with a configurable delay period (e.g., 2 days), then executing the change after the delay expires. The delay provides time for monitoring systems to detect unauthorized change attempts while still enabling legitimate key rotation for security hygiene or incident response.

Resolution: Mitigated:

`swapSigner` remains `immutable` in **DexalotRFQ**, and no on-chain signer-rotation mechanism is present. Key-rotation and compromise response are handled through operational replacement of the contract and fund migration, rather than in-contract mutability or delayed rotation controls.

Client comment:

This is an intentional design decision for a more gas optimal immutable DexalotRFQ contract. Since this contract is used by DeFi aggregators gas optimisation is a priority. In terms of key rotation we will deploy a fresh contract with new key and transfer funds across using the rebalancer role, deprecating the old contract. In the case of key compromise we have scripts and tooling to transfer funds from the RFQ contract to the rebalancer role in situations where the contract seems to be in the process of draining. Once recovered we would then deploy a fresh contract with new swapSigner key

[F-2026-14880](#) - Silent Failure in matchAuctionOrder Causes Infinite Loop in matchAuctionOrders - Low

Description:

A recent code change in **TradePairs**.`matchAuctionOrder` replaced `require` statements with an `if` condition to save contract size. This modification introduced a silent failure mode that can cause an infinite loop when called from **ExchangeSub**.`matchAuctionOrders`.

```
function matchAuctionOrder(...) returns (uint256 quantityRemaining) {
    TradePair storage tradePair = tradePairMap[_takerOrder.tradePairId];
    if (tradePair.auctionMode == AuctionMode.MATCHING && tradePair.auctionPrice > 0) {
        (_takerOrder, ) = matchOrder(_takerOrder, _maxNbrOfFills, STP.NONE);
        quantityRemaining = UtilsLibrary.getRemainingQuantity(_takerOrder.quantity, _takerOrder.quantityFilled);
    }
    // If conditions are NOT met, quantityRemaining defaults to 0
}
```

The code change includes a developer comment: "`Commented out require statements to save room in contract size CD 2025-07-01. Auction functionality will not be used for the foreseeable future.`" However, the auction functionality remains accessible to accounts with `AUCTION_ADMIN_ROLE`, and the code change introduces incorrect behavior.

Caller Logic in `matchAuctionOrders` function of **ExchangeSub** contract:

```
uint256 startRemainingQuantity = UtilsLibrary.getRemainingQuantity(
    takerOrder.quantity,
    takerOrder.quantityFilled
);
uint256 takerRemainingQuantity = tradePairs.matchAuctionOrder(takerOrder, _maxNbrOfFills);
if (startRemainingQuantity == takerRemainingQuantity) {
    // No fills occurred - auction finished
    emit AuctionMatchFinished(_tradePairId);
    return false;
} else {
    // Fills occurred - continue matching
    return true;
}
```

When `matchAuctionOrder` is called with incorrect preconditions (`auctionMode != MATCHING` or `auctionPrice == 0`):

1. The function skips matching and returns `quantityRemaining = 0` (Solidity's default for uint256)
2. The caller (`matchAuctionOrders`) interprets 0 as "order was completely filled"
3. Since `startRemainingQuantity` (e.g., 80) $\neq$ `takerRemainingQuantity` (0), it returns `true`
4. The caller believes matching occurred and more orders remain to be processed
5. On the next call, the same unmodified order is still at the top of the orderbook
6. The cycle repeats indefinitely

This leads to:

1. **Gas Exhaustion:** Admin repeatedly calling the function wastes gas without achieving any state changes
2. **Operational Confusion:** No clear indication why auction matching never completes
3. **Stuck Auction State:** The trade pair may remain locked in an incorrect auction mode indefinitely
4. **Misleading Return Value:** The function returns true (indicating more work to do) when no actual matching is possible

Assets:

- contracts/TradePairs.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Return original remaining quantity on precondition failure:

```
function matchAuctionOrder(  
    Order memory _takerOrder,  
    uint256 _maxNbrOfFills
```

```

) external override onlyRole(DEFAULT_ADMIN_ROLE) returns (uint256 quantityRem
aining) {
    TradePair storage tradePair = tradePairMap[_takerOrder.tradePairId];

    // Default to original remaining quantity (no change)
    quantityRemaining = UtilsLibrary.getRemainingQuantity(
        _takerOrder.quantity,
        _takerOrder.quantityFilled
    );

    if (tradePair.auctionMode == AuctionMode.MATCHING && tradePair.auctionPric
e > 0) {
        (_takerOrder, ) = matchOrder(_takerOrder, _maxNbrOfFills, STP.NONE);
        quantityRemaining = UtilsLibrary.getRemainingQuantity(
            _takerOrder.quantity,
            _takerOrder.quantityFilled
        );
    }
}

```

Resolution:

Fixed in [361d00f](#):

The issue has been resolved by deprecating the `matchAuctionOrder` function entirely. The function now performs no matching operations and returns `_takerOrder.quantity` (the full order quantity).

```

* @notice Deprecate function to match Auction orders
* @dev      Performs no action as auction matching logic is deprecated.
...
function matchAuctionOrder(
    Order memory _takerOrder,
    uint256
) external view override onlyRole(DEFAULT_ADMIN_ROLE) returns (uint256 quanti
tyRemaining) {
    return _takerOrder.quantity;
}

```


[F-2026-14882](#) - Removal of Swap Expiry Mechanism Prevents Server-Side Quote Invalidation - Low

Description:

The **DexalotRFQ** contract removes the `updateSwapExpiry` function and `expiredSwaps` mapping that existed in **MainnetRFQ**, eliminating the ability to invalidate quotes before their natural expiry. This contradicts the contract's own documentation which references the `SwapExpired` event for adjusting quotes during high volatility periods.

The contract documentation explicitly mentions this capability:

```
* Monitor the SwapExpired event to verify if a swap has been adjusted. Adjusting the quote is rare, and
* only resorted to in periods of high volatility for quotes that do not properly represent the liquidity
* of the Dexalot L1.
```

However, the mechanism to emit this event has been removed. The previous **MainnetRFQ** implementation included:

```
function updateSwapExpiry(uint256 _nonceAndMeta) external onlyRole(REBALANCER_ADMIN_ROLE) {
    expiredSwaps[_nonceAndMeta >> 8] |= 1 << (_nonceAndMeta & 0xff);
    emit SwapExpired(_nonceAndMeta, block.timestamp);
}
```

This was verified during swap execution:

```
require(expiredSwaps[bucket] & mask == 0, "RF-QE-01");
```

The new **DexalotRFQ** contract contains neither the `expiredSwaps` mapping, the `updateSwapExpiry` function, nor the verification check. Once a quote is signed, it remains valid until its natural expiry timestamp regardless of market conditions or security concerns.

This leads to:

- **Stale quote exploitation:** During periods of high volatility, quotes that no longer represent fair market value cannot be invalidated. Users can execute outdated favorable quotes after significant market moves.
- **Documentation inconsistency:** The contract documentation references functionality that does not exist, potentially misleading integrators.

Assets:

- contracts/DexalotRFQ.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status:

Mitigated

Classification**Impact Rate:**

3/5

Likelihood Rate:

2/5

Exploitability:

Independent

Complexity:

Simple

Severity:

Low

Recommendations**Remediation:**

The swap expiry mechanism should be re-implemented to maintain the operational capability described in the contract documentation.

Alternatively, if this is an intentional trade-off for the non-upgradeable design, shorter quote expiry times should be enforced to reduce the exposure window, and the documentation should be updated to accurately reflect the contract's capabilities.

Resolution:

Mitigated in `243eca9`:

Risk exposure is reduced through updated slippage handling in **DexalotRFQ** and documentation alignment that removes the obsolete `SwapExpired` monitoring statement.

Client comment:

This is an intentional design decision as aggregators block flow if we revert on chain. To combat this we rely on slippage curves which slip a user based on volatility and how long it takes them to execute. In addition we currently have quote expiry times of 30s to reduce the exposure window

[F-2026-14883](#) - Missing msg.value Validation for Native Token Swaps

- Low

Description:

The **DexalotRouter** contract facilitates token swaps by forwarding calls to allowed RFQ contracts. When performing swaps involving native tokens (ETH/AVAX), the contract does not validate that the `msg.value` sent by the user matches the expected `takerAmount` specified in the swap parameters.

In `multiPartialSwap` function:

```
function multiPartialSwap(
    IDexalotRFQ.Order calldata _orderA,
    bytes calldata _signatureA,
    uint256 _takerAmountA,
    IDexalotRFQ.Order calldata _orderB,
    bytes calldata _signatureB
) external payable {
    // ...

    if (_orderA.takerAsset != address(0)) {
        IERC20(_orderA.takerAsset).safeTransferFrom(msg.sender, _orderA.maker,
            _takerAmountA);
    }
    // If takerAsset == address(0) (native token), no transfer occurs here
    // The native token is forwarded via msg.value below

    // ...

    (bool success, ) = payable(_orderA.maker).call{value: msg.value}(_orderA.callDataA);
    // ^^ msg.value is forwarded without validating it equals _takerAmountA
```

In `fallback` function:

```
fallback() external payable {
    // ...

    address takerAsset;
    uint256 amount;
    assembly {
        takerAsset := calldataload(TAKER_ASSET_OFFSET)
        amount := calldataload(amountOffset)
    }
}
```

```

if (takerAsset != address(0)) {
    IERC20(takerAsset).safeTransferFrom(msg.sender, targetImplementation,
amount);
}
// If takerAsset == address(0), no validation that msg.value == amount

(bool success, ) = targetImplementation.call{value: msg.value}(newCallData
);
// ^^ msg.value is forwarded without validation

```

Scenario 1: User Sends Excess Native Tokens

1. User initiates a native token swap with `takerAmount` = 1 ETH
2. User accidentally sends `msg.value` = 2 ETH
3. The RFQ contract processes the swap for 1 ETH
4. The extra 1 ETH remains stuck in the **DexalotRouter** contract
5. User loses 1 ETH (can only be recovered by admin via `retrieveToken`)

Scenario 2: User Sends Insufficient Native Tokens

1. User initiates a native token swap with `takerAmount` = 1 ETH
2. User sends `msg.value` = 0.5 ETH
3. The call to RFQ contract fails, but the error message is propagated from the downstream contract
4. User experiences confusing error messages and wasted gas

Scenario 3: User Sends Native Tokens for ERC20 Swap

1. User initiates an ERC20 swap (`takerAsset` is an ERC20 token)
2. User mistakenly includes `msg.value` = 1 ETH in the transaction
3. The ERC20 transfer proceeds normally
4. The 1 ETH sent is forwarded to the RFQ contract unnecessarily
5. ETH may be stuck in the RFQ contract or cause unexpected behavior

Assets:

- `contracts/DexalotRouter.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Low

Recommendations

Remediation: Add explicit validation for `msg.value` in both functions.

Resolution: Fixed in `229271a`:

The reported scenarios have been addressed through different approaches:

Scenario 1:

Client Response:

Scenario 1 is allowable, funds are not stuck in DexalotRouter but fully forwarded onto the child DexalotRFQ contract. The child contract consumes the excess and returns the makerAmount of the quote. The excess is then refunded by the child contract in `_refundNative`."

Scenario 2:

The child RFQ contract reverts if there are insufficient native tokens for the swap

Scenario 3:

This scenario has been addressed in a separate fix (**F-2026-14915**)

```
require(msg.value == 0, "DR-NFES-01");
```

F-2026-14891 - Switching to bulkTransferTokens Removes AutoFill Functionality for Claim Recipients - Low

Description:

The updated **IncentiveDistributor** contract uses `portfolio.bulkTransferTokens` for claim transfers instead of the previous `transferToken` calls. This change removes the automatic gas tank fill functionality that recipients previously received, as `bulkTransferTokens` does not invoke `autoFillPrivate` for the recipient.

In the previous implementation, each claim used the public `transferToken(to, symbol, quantity)` function on **PortfolioSub**. This function performs the transfer and then calls `autoFillPrivate(_to, _symbol, Tx.AUTOFILL)`, providing recipients with automatic gas tank fill behavior.

The updated claim function uses `bulkTransferTokens`:

```
portfolio.bulkTransferTokens(address(this), msg.sender, symbols, amounts);
```

The `bulkTransferTokens` function in **PortfolioSub** only calls the private `transferToken(_from, _to, symbol, quantity, ...)` in a loop and does not call `autoFillPrivate` for the recipient. This means claim recipients in the updated version no longer receive the `AutoFill` behavior after claiming rewards.

The change affects the expected user experience for the Dexalot Incentive Program (DIP) rewards distribution, where users claiming ALOT tokens would previously have their gas tanks automatically topped up.

Users and vaults that expect automatic gas top-up (e.g., ALOT swapped for gas) after claiming rewards will not receive it.

Assets:

- contracts/token/IncentiveDistributor.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple

Severity:

Low

Recommendations**Remediation:**

If `AutoFill` for claim recipients is intended behavior, a call to trigger `AutoFill` for the claimer should be added after `bulkTransferTokens` completes. This could be achieved by adding a dedicated post-bulk-transfer `AutoFill` hook on **PortfolioSub**, or by adding a variant of `bulkTransferTokens` that optionally runs `autoFillPrivate` for the recipient.

If the change is intentional for gas savings or because claimers are expected to manage gas themselves, this should be clearly documented.

Resolution:

Fixed in `bf20793`:

`bulkTransferTokens` in **PortfolioSub** now invokes `autoFillPrivate(_to, _symbols[0], Tx.AUTOFILL)` after the transfer loop.

Claim transfers from **IncentiveDistributor** continue to use `portfolio.bulkTransferTokens(...)`, and `AutoFill` behavior for recipients is restored through the added post-bulk-transfer hook.

F-2026-14896 - Use of Deprecated transfer Function for Native Token Withdrawal - Low

Description:

The `withdrawGas` function uses the deprecated `transfer` method to send native tokens, which forwards a fixed 2300 Gas stipend. This may cause withdrawals to fail if the owner address is a contract with a `receive` function requiring more gas.

The `withdrawGas` function uses transfer for sending native tokens:

```
function withdrawGas(uint256 amount) external onlyOwner {  
    require(address(this).balance >= amount, "ID-AGCB-01");  
    emit WithdrawGas(msg.sender, amount, block.timestamp);  
    payable(msg.sender).transfer(amount);  
}
```

The `transfer` function forwards exactly 2300 gas to the recipient. While sufficient for EOA addresses and simple receive functions, this fixed stipend may be insufficient for:

- Multisig wallets with complex receive logic
- Smart contract wallets that perform logging or state updates on receive
- Future changes to gas costs that could make 2300 gas insufficient for basic operations

Assets:

- contracts/token/IncentiveDistributor.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

The `transfer` call should be replaced with a low-level `call` with proper success validation:

```
(bool success, ) = msg.sender.call{value: amount}("");  
require(success, "ID-WTFR-01");
```

This pattern forwards sufficient gas for any receiver implementation and follows current best practices for native token transfers.

Resolution:

Fixed in `d0365fe`:

`withdrawGas` in **IncentiveDistributor** was updated from `transfer(amount)` to low-level `call{value: amount}("")` with explicit success validation (`ID-WTFR-01`).

[F-2026-14900](#) - Semantic Storage Variable Change Requires Migration During InventoryManager Upgrade - Low

Description:

The `scalingFactor` mapping retains the same storage layout between contract versions but has a fundamentally different semantic meaning. Old values represent decimal normalizers (divisors like `10^12`), while new values represent target chain ratios in basis points (`BPS`). If tokens are not migrated during upgrade, fee calculations will be incorrect.

The storage variable interpretation differs between versions:

Version Purpose		Example Value	Meaning
Old	Decimal normalizer (divisor)	<code>10^12</code>	Divide inventory by <code>10^12</code>
New	Target ratio in BPS	<code>3333</code>	Chain should hold 33.33% of total

If a token previously had `scalingFactor = 10^12` (for decimal normalization), after upgrade the new code interprets this as `targetChainRatio = 10^12` in `BPS`. Since `BPS` maximum is `10000`, having `10^12` causes the `divBPS` calculation to produce near-zero ratios, leading to drastically incorrect fee calculations. The `setScalingFactors` function allows overwriting values:

```
function setScalingFactors(
    bytes32[] memory _symbolIds,
    uint16[] memory _scalingFactors
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    for (uint256 i = 0; i < _symbolIds.length; ++i) {
        // ...
        scalingFactor[symbolId] = sf;
        emit ScalingFactorUpdated(symbolId, sf, block.timestamp);
    }
}
```

This can lead to:

- **Incorrect fee calculations:** Tokens with unmigrated scaling factors will have fees calculated using misinterpreted values, potentially resulting in near-zero or excessively high fees.
- **Partial migration risk:** If any token's scaling factor is not updated during the upgrade window, that token's fee calculations remain incorrect until manually fixed.

Assets:

- contracts/InventoryManager.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status:

Mitigated

Classification

Impact Rate:

5/5

Likelihood Rate:

2/5

Exploitability:

Dependent

Complexity:

Simple

Severity:

Low

Recommendations

Remediation:

Document the required migration steps in the upgraded `InventoryManager`.

- **Before the upgrade window:**

- Query all `symbolIds` with a non-zero `scalingFactor` prior to the upgrade.
- Prepare the corresponding new BPS-based values for each identified `symbolId`.

- **During the upgrade:**

- Execute `setScalingFactors` immediately after the upgrade and **before** unpausing the system.

Resolution:

Mitigated:

The risk is reduced through an operational migration procedure, while contract-level enforcement remains absent for legacy-to-BPS `scalingFactor` semantic transition.

Client comment:

Agreed, we have scripts in place which first query all the `symbolIds` to confirm the `scalingFactor` values on chain match what we have stored on the database. We will then pause PortfolioBridge so deposits/withdrawals are blocked so inventory in InventoryManager cannot change. Then we will perform the upgrade. We will then call `setScalingFactors` for all `symbolIds` with the new BPS format. Finally we will unpause the system

F-2026-14901 - Division by Zero When Withdrawing Total Remaining Inventory - Low

Description:

The `calculateWithdrawalFee` function reverts with a division by zero error when a user attempts to withdraw the entire remaining total inventory across all chains. The formula divides by `( _totalInventory - _quantity )`, which becomes zero when withdrawing the full amount.

The fee calculation in **InventoryFeeCalculatorLibrary** computes the new chain ratio:

```
uint256 newChainRatio = (BPS * (_chainInventory - _quantity)) / (_totalInventory - _quantity);
```

The `calculateWithdrawalFee` function in **InventoryManager** only validates that the withdrawal amount does not exceed the current chain's inventory:

```
require(currentInventory >= _withdrawal.quantity, "IM-INVT-02");
```

However, there is no validation that the total inventory across all chains exceeds the withdrawal quantity. If `totalInventory == _withdrawal.quantity`, the denominator becomes zero, causing a revert with a generic panic error rather than a meaningful message.

This scenario can occur when:

- A single chain holds all remaining inventory and a user withdraws the full amount
- Rebalancing operations result in concentrated inventory on one chain

This can lead to:

- **Denial of service:** The last withdrawal that would drain total inventory always fails with a cryptic division by zero error.
- **Poor error handling:** Users receive a generic panic error instead of a meaningful message explaining why the withdrawal cannot be processed.
- **Rebalancing edge cases:** May affect edge cases during rebalancing operations where inventory becomes concentrated.

Assets:

- `contracts/InventoryManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]
- `contracts/library/InventoryFeeCalculatorLibrary.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Independent

Complexity: Simple

Severity: Low

Recommendations

Remediation: An explicit check should be added in `calculateWithdrawalFee` before calling the library function:

```
if (totalInventory == _withdrawal.quantity) {  
    return 0; // No fee for final withdrawal, or revert with meaningful error  
}
```

Alternatively, if withdrawing all inventory should be prohibited, a descriptive require statement should be added to provide clear feedback to users.

Resolution: Fixed in `4cdc9ed`:

`calculateWithdrawalFee` now guards the denominator edge case by returning zero fee when `totalInventory == _withdrawal.quantity` before calling the fee library:

```
// No fee is withdrawing total inventory across all chains  
if (totalInventory == _withdrawal.quantity) {  
    return 0;  
}
```

This removes the division-by-zero revert path and aligns behavior with explicit handling of full-inventory withdrawals.

F-2026-14905 - K Value Not Validated as Multiple of 4 Causes Incorrect Fee Calculation - Low

Description:

The `updateFutureK` function validates only that the `K` value is within the allowed range but does not validate that it is a multiple of 4. Since `powMultipleOf4BPS` is designed for multiples of 4 (8, 12, 16, 20, 24, 28, 32), non-conforming values default to `K=32`, causing higher-than-expected fee sensitivity.

The `updateFutureK` function only validates the `K` value range:

```
function updateFutureK(uint256 _K, uint256 _timePeriod) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_K >= MIN_K && _K <= MAX_K, "IM-KVNP-01");
    require(_timePeriod >= MIN_K_UPDATE_TIME, "IM-KTNP-01");
    futureK = _K;
    futureKTime = block.timestamp + _timePeriod;
    emit FutureKUpdated(_K, futureKTime, block.timestamp);
}
```

The `powMultipleOf4BPS` function in `InventoryFeeCalculatorLibrary` handles specific multiples of 4:

```
if (K == 8) {
    return R8;
}
if (K == 12) {
    return mulBPS(R8, R4);
}
// ... other cases for 16, 20, 24, 28 ...
return mulBPS(R16, R16); // Default: R^32
```

If `K` is set to a non-multiple of 4 (e.g., 9, 10, 11, 13), the function falls through to the default case and returns `R^32` (maximum exponent). This means an admin intending to set moderate fee sensitivity with `K=10` would inadvertently apply maximum sensitivity.

Assets:

- `contracts/InventoryManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation: Validation should be added to `updateFutureK` to ensure `K` is a multiple of 4:

```
require(_K % 4 == 0, "IM-KMOD-01"); // K must be multiple of 4
```

This ensures only valid `K` values that `powMultiple0f4BPS` explicitly handles can be set, preventing silent fallback to the default case.

Resolution: Fixed in `3d30068`:

`updateFutureK` now enforces multiple-of-4 validation with:

```
require(_K % 4 == 0, "IM-KMOD-01");
```

This prevents unsupported `K` values from reaching the library fallback path and aligns runtime behavior with `powMultiple0f4BPS` design constraints.

[F-2026-14907](#) - calculateWithdrawalFee Returns Zero Fee for Zero Inventory Instead of Reverting - Low

Description:

The `calculateWithdrawalFee` function returns a zero fee when `currentInventory == 0` due to an early return that bypasses the inventory validation check. This causes fee quotes to indicate a withdrawal is possible with low fees when the actual withdrawal will revert due to insufficient inventory.

The function has an early return before the inventory validation:

```
function calculateWithdrawalFee(
    IPortfolioBridgeSub.XferShort calldata _withdrawal
) external view returns (uint256 fee) {
    // ...
    uint256 currentInventory = get(_withdrawal.symbol, _withdrawal.symbolId);

    if (numChains == 1 || currentInventory == 0) {
        return 0; // Early return - bypasses validation below
    }

    require(currentInventory >= _withdrawal.quantity, "IM-INVT-02"); // Never
    // reached when currentInventory == 0
    // ...
}
```

When `currentInventory == 0` and `_withdrawal.quantity > 0`, the function returns `0` (indicating no fee and implying the withdrawal is possible) instead of reverting.

This affects downstream functions in **PortfolioBridgeSub**:

- In `getAllBridgeFees`, the try-catch is designed to skip chains where withdrawal is impossible by catching reverts. Since `calculateWithdrawalFee` returns `0` instead of reverting, the chain appears available with low fees when inventory is actually zero.
- In `_calcBridgeFee`, a valid-looking bridge fee is returned (with 0 inventory component) for an impossible withdrawal.

When the user actually attempts the withdrawal, the decrement function correctly reverts with `"IM-INVT-01"` due to insufficient inventory, contradicting the fee quote.

This lead to:

- **Misleading fee quotes:** Users receive fee quotes suggesting withdrawals are possible and cheap when they will actually fail.

- **Incorrect chain availability:** `getAllBridgeFees` includes chains with zero inventory as valid withdrawal destinations.
- **Poor user experience:** Users may attempt withdrawals based on fee quotes only to have transactions revert.

Assets:

- `contracts/InventoryManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

The inventory validation should be moved before the early return to ensure consistency:

```
uint256 currentInventory = get(_withdrawal.symbol, _withdrawal.symbolId);

// Check BEFORE early return
require(currentInventory >= _withdrawal.quantity, "IM-INVT-02");

if (numChains == 1 || currentInventory == 0) {
    return 0;
}
```

This ensures that withdrawals exceeding available inventory revert consistently in both fee calculation and actual withdrawal execution.

Resolution:

Fixed in `d75e9b4`:

`calculateWithdrawalFee` now validates inventory sufficiency before any zero-fee early return:

- `require(currentInventory >= _withdrawal.quantity, "IM-INVT-02");` executes first.
- The `currentInventory == 0` early-return path was removed.

- Zero-fee return remains only for `numChains == 1`, after inventory validation.

Fee quotation behavior is now consistent with withdrawal execution constraints, and zero-inventory chains no longer appear as valid low-fee withdrawal routes.

[F-2026-14910](#) - Unenforced Invariant Between minTradeAmount and minPostAmount Causes Unexpected Order Cancellations - Low

Description:

The interface documentation states that `minTradeAmount` should be "less than or equal to the `minPostAmount`", but the setter functions do not enforce this relationship. When `minPostAmount` exceeds `minTradeAmount`, orders that pass the entry validation can have their remaining quantity automatically canceled after partial fills, causing unexpected behavior for users.

The **TradePair** struct documentation in **ITradePairs.sol** establishes the intended invariant:

```
@param minTradeAmount minimum trade amount as a taker. Less than or equal to the minPostAmount
```

However, both `setMinPostAmount` and `setMinTradeAmount` allow administrators to set values that violate this relationship:

```
function setMinPostAmount(
    bytes32 _tradePairId,
    uint256 _minPostAmount
) external override onlyRole(DEFAULT_ADMIN_ROLE) {
    TradePair storage tradePair = tradePairMap[_tradePairId];
    uint256 oldValue = tradePair.minPostAmount;
    tradePair.minPostAmount = _minPostAmount;
    emit ParameterUpdated(PARAMETER_UPDATED_VERSION, _tradePairId, "T-MINPOAM", oldValue, _minPostAmount);
}

function setMinTradeAmount(
    bytes32 _tradePairId,
    uint256 _minTradeAmount
) external override onlyRole(DEFAULT_ADMIN_ROLE) {
    TradePair storage tradePair = tradePairMap[_tradePairId];
    uint256 oldValue = tradePair.minTradeAmount;
    tradePair.minTradeAmount = _minTradeAmount;
    emit ParameterUpdated(PARAMETER_UPDATED_VERSION, _tradePairId, "T-MINTRAM", oldValue, _minTradeAmount);
}
```

In `addOrderPrivate`, remaining order quantities below `minPostAmount` are automatically canceled with status code `T-LTPA-01`. When `minPostAmount > minTradeAmount`, a scenario emerges where:

- A user submits an order that passes the `minTradeAmount` entry check
- The order partially fills, leaving a remainder
- The remainder exceeds `minTradeAmount` but falls below `minPostAmount`
- The remainder is silently canceled instead of being posted to the orderbook

This can lead to:

- **Unexpected cancellations:** Orders that legitimately pass entry validation can have their remaining quantities canceled after partial fills.
- **Lost trading opportunities:** Users may lose the ability to have their remaining order quantity filled.
- **Confusion:** The behavior contradicts user expectations based on the `minTradeAmount` they were required to meet.

Example scenario: With `minTradeAmount = $10` and `minPostAmount = $50`, a user submits a `$100` order. It matches for `$60`, leaving `$40`. The `$40` remainder is canceled even though it exceeds the documented minimum trade amount.

Assets:

- `contracts/TradePairs.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

- **Option A - Enforce the Documented Invariant:**
 - Add validation to both setter functions to ensure `minTradeAmount <= minPostAmount` is always maintained. `setMinPostAmount` should require the new value is greater than or equal to current `minTradeAmount`, and `setMinTradeAmount` should require the new value is less than or equal to current `minPostAmount`.
- **Option B - Update Documentation:**

- If the flexibility to set any values is intentional, update the NatSpec documentation to remove or clarify the "less than or equal" statement and add comprehensive documentation explaining the user impact when `minPostAmount > minTradeAmount`, specifically that remaining quantities below `minPostAmount` will be auto-canceled.

Resolution:

Mitigated:

The on-chain invariant `minTradeAmount <= minPostAmount` remains unenforced in `setMinPostAmount` and `setMinTradeAmount`, while interface documentation continues to define this relationship as an expected constraint. Operational safeguards are documented in off-chain deployment and setup processes, but no on-chain validation enforces the invariant.

Client comment:

An explicit require check was intentionally omitted from `setMinPostAmount + setMinTradeAmount` as `TradePairs.sol` is near the maximum size capacity. We mitigate this with our deployment/setup scripts which have explicit checks to ensure `minTradeAmount ≤ minPostAmount` always holds, this is done in both our database upsert and hardhat scripts

[F-2026-14914](#) - Immutable trustedForwarder Without Revocation Capability - Low

Description: The `trustedForwarder` address in **DexalotRFQ** is set as immutable during contract deployment and cannot be changed or revoked if compromised, creating a permanent security risk.

```
address public immutable trustedForwarder;

constructor(..., address _trustedForwarder
) ... {
    ...
    trustedForwarder = _trustedForwarder;
}
```

How `trustedForwarder` is used:

```
function _msgSender() internal view virtual override returns (address) {
    if (msg.data.length >= ADDRESS_LENGTH && msg.sender == trustedForwarder) {
        return address(bytes20(msg.data[msg.data.length - ADDRESS_LENGTH:
]));
    } else {
        return msg.sender;
    }
}
```

The `trustedForwarder` has special privileges to specify arbitrary sender addresses by appending them to calldata. This is used by **DexalotRouter** to forward user transactions.

If the `trustedForwarder` contract (e.g., **DexalotRouter**) is compromised through:

- Upgrade vulnerability (if upgradeable)
- Admin key compromise
- Logic bug exploitation

Consequences:

- Attacker can impersonate any user address
- Execute swaps on behalf of any user who has token approvals to **DexalotRFQ**
- Drain user funds through malicious swap orders

Mitigation impossible: Cannot revoke or change `trustedForwarder` without deploying entirely new contract.

Assets:

- `contracts/DexalotRFQ.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Consider making `trustedForwarder` mutable with admin control or adding emergency disable flag.

Resolution: The issue has been acknowledged by the development team.

Client Response:

Intentional decision to save on gas. If there are issues our first step is to pause quoting from our off-chain component so even if our `trustedForwarder` is compromised an attacker has no attack surface as there are no quotes. To mitigate having to fully redeploy all child RFQ contracts we will shift `DexalotRouter` to be upgradable with UUPS

[F-2026-14939](#) - Token Removal Migration Leaves Orphaned Data in tokenInfoMapBySymbolChainId - Low

Description:

In version **4.1.6** and earlier, the `removeToken` function in the **PortfolioBridgeSub** deleted an entry using the different mapping key:

```
delete (tokenInfoMapBySymbolChainId[_srcChainSymbol][_srcChainId]);
```

Currently it uses `_subnetSymbol` instead of `_srcChainSymbol`:

```
delete (tokenInfoMapBySymbolChainId[_subnetSymbol][_srcChainId]);
```

Every token removed using version **4.1.6** or earlier left behind orphaned data in `tokenInfoMapBySymbolChainId`. The entry at `[_subnetSymbol][_srcChainId]` was never deleted.

Orphaned entries permanently occupy storage slots, increasing the contract's storage footprint.

Assets:

- contracts/PortfolioBridgeSub.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Accepted

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Consider manually cleaning any orphaned entries from tokens that were removed using the old version:

```
function cleanOrphanedTokenInfo(  
    bytes32 _subnetSymbol,
```



```

uint32 _srcChainId
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    require(tokenInfoMapBySymbolChainId[_subnetSymbol][_srcChainId].symbolId !=
bytes32(0), "No orphan");
    bytes32 symbolId = tokenInfoMapBySymbolChainId[_subnetSymbol][_srcChainId]
.symbolId;
    require(!tokenListById.contains(symbolId), "Token still active");
    delete tokenInfoMapBySymbolChainId[_subnetSymbol][_srcChainId];
}

```

Resolution:

The issue has been acknowledged by the development team.

Client Response:

In production it is not required to cleanup as we have not ran into token removal yet. For token removal to occur we need all deposits to be withdrawn after a pair is delisted, since user's typically forget and leave dust amounts we have not been able to remove tokens from PortfolioBridgeSub even if they've been delisted.

[F-2026-14948](#) - Cross-Chain Message Loss During Contract Upgrade Due to Payload Format Incompatibility - Low

Description:

The upgrade from **v4.1.6** to **v4.2.0** introduces a breaking change in the **CCTRADE** message payload format. The new version appends the sender's RFQ contract address (20 bytes) to **CCTRADE** payloads, changing the expected payload length from 128 bytes to 148 bytes. This incompatibility causes all in-flight **CCTRADE** messages to fail during the upgrade window.

Old version (**v4.1.6**) - **packXferMessage**:

```
function packXferMessage(IPortfolio.XFER memory _xfer) private pure returns (
bytes memory message) {
    // ... packing logic ...
    message = bytes.concat(slot0, slot1, slot2, slot3); // 128 bytes total
}
```

New version (v4.2.0) - **packXferMessage**:

```
function packXferMessage(IPortfolio.XFER memory _xfer) private view returns (
bytes memory message) {
    // ... packing logic ...
    if (_xfer.transaction == IPortfolio.Tx.CCTRADE) {
        bytes20 slot4 = bytes20(msg.sender); // RFQ address appended
        message = bytes.concat(slot0, slot1, slot2, slot3, slot4); // 148 bytes
    } else {
        message = bytes.concat(slot0, slot1, slot2, slot3); // 128 bytes
    }
}
```

New version (**v4.2.0**) - **unpackXferMessage**:

```
function unpackXferMessage(bytes calldata _payload)
external pure returns (IPortfolio.XFER memory xfer, address senderAddress)
{
    // ... unpacking logic ...
    if (xfer.transaction == IPortfolio.Tx.CCTRADE) {
        require(_payload.length == 148, "PB-UM-02"); // Strict length check!
        senderAddress = address(bytes20(_payload[128:148]));
    } else {
        require(_payload.length == 128, "PB-UM-03");
    }
}
```

```
}  
}
```

Scenario: Old Chain → New Chain (during upgrade)

1. User's source tokens (e.g., 10,000 USDC) are locked in **DexalotRFQ** on Chain A
2. Message arrives at Chain B but `processPayload` reverts due to length mismatch
3. User receives nothing on Chain B
4. Funds are stuck - no automatic refund mechanism exists

Assets:

- contracts/PortfolioBridgeMain.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate:	4/5
Likelihood Rate:	2/5
Exploitability:	Semi-Dependent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Document and enforce a safe upgrade procedure:

1. Pause **PortfolioBridgeMain** on all chains simultaneously
2. Wait for drain period to ensure all in-flight messages complete
3. Verify no pending messages in bridge queues
4. Upgrade all chains simultaneously
5. Unpause all chains

Resolution:

This issue was mitigated through documented operational procedures for the upgrade process:

Client Response:

CCTrade messages are only generated by xChainSwaps which are generated and signed by an off-chain component. This component sets a 30 second expiry time for any quote it generates. Therefore for the upgrade we will pause giving out any

xChainSwap quotes, after 30 seconds we will wait for all in flight messages to complete. Once we've verified there are no pending messages we will upgrade the contract chain by chain, once all chains are complete we can reenale our off-chain component. This process means we won't need to upgrade all chains simultaneously (as that will be difficult to coordinate).

F-2026-15243 - Defensive Check Ordering in `_calcSharesToMint` Allows Theoretical Division by Zero - Low

Description:

The `_calcSharesToMint` function performs a division before checking whether `totalShares == 0`:

```
uint256 totalShares = _tloadVaultTotalShares(_vaultId);
sharesToMint = (userDepositUsd * totalShares) / _tloadVaultUSD(_vaultId);
if (totalShares == 0) {
    sharesToMint = userDepositUsd;
}
```

When `totalShares == 0`, the numerator evaluates to 0. If `_tloadVaultUSD(_vaultId)` is also 0, the expression becomes `0 / 0`, which triggers a Solidity 0.8+ arithmetic panic and reverts the entire `bulkSettleState` transaction. The preconditions for reaching this state require:

- All vault shares to have been fully withdrawn and burned in prior batches (`totalSupply == 0`)
- The settler to provide a vault state with zero total USD value at finalization time

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	4/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Move the `totalShares == 0` guard before the division to prevent any possibility of a zero-denominator panic:

```
uint256 totalShares = _tloadVaultTotalShares(_vaultId);
if (totalShares == 0) {
    return userDepositUsd;
}
sharesToMint = (userDepositUsd * totalShares) / _tloadVaultUSD(_vaultId);
```

Resolution:

Fixed in [525771d](#):

The `totalShares == 0` guard was moved before the division:

```
if (totalShares == 0) {
    return userDepositUsd;
}
return (userDepositUsd * totalShares) / _tloadVaultUSD(_vaultId);
```

[F-2026-16203](#) - [DualDefense] Permissionless UnwindBatch Can Dkip Batch Ordering And Permanently Lock Finalized Withdrawal Requests - Low

Description:

An attacker can permissionlessly call `unwindBatch([], [])` on the current batch to advance `currentBatchId` and skip over a previously `FINALIZED` batch containing real user withdrawal requests. Because settlement is hard-pinned to `currentBatchId - 1`, the skipped finalized batch becomes unreachable for valid settlement.

OmniVaultManager combines:

1. A settlement pointer that only allows processing `currentBatchId - 1`.
2. A permissionless unwind path that can advance `currentBatchId` even with empty inputs.

This allows non-privileged callers to move the batch pointer forward and strand previously finalized requests.

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Accepted

Classification

Impact Rate:	3/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Mirror the ordering invariant that `finalizeBatch` already enforces on its predecessor. A batch should only be unwindable if every earlier batch is already `SETTLED` or `UNWOUND`, so that unwinding the current batch can never leapfrog an unsettled `FINALIZED` predecessor.

Resolution:

No on-chain code change addresses this finding. The `unwindBatch` function remains permissionless, accepts empty arrays against zeroed rolling hashes, and unconditionally advances `currentBatchId` via `_resetBatch()`. The team acknowledged the finding as an acceptable operational risk: exploitation requires the settler to have already failed its settlement SLA for the full 24-hour `RECLAIM_DELAY` window, the attack is pure griefing with no economic incentive, and the upgradeable proxy architecture allows administrative recovery of stranded batches if needed.

[F-2026-16204](#) - [DualDefense] finalizeBatch Can Never Finalize Batch Zero Because It Reads batchId - 1 Before The Zero-Batch Guard - Low

Description:

`finalizeBatch` is intended to permit the first batch to finalize even though there is no prior batch. The code attempts to encode that rule with `batchId == 0 || prevBatchStatus == ...`, but it computes `prevBatchStatus` using `completedBatches[batchId - 1]` before the guard is evaluated.

Because `currentBatchId` starts at `0`, the subtraction underflows on the first settlement attempt and the function reverts with `Panic(0x11)`. The vulnerable path is triggered on the very first normal settlement cycle. `currentBatchId` begins at `0`, so the first authorized settler call into `finalizeBatch` executes `completedBatches[batchId - 1].status`.

In Solidity 0.8.x, `0 - 1` causes a checked-arithmetic revert before the later `batchId == 0 || ...` expression can short-circuit. That means the intended batch-zero exception is dead code for the first call.

Assets:

- contracts/vaults/OmniVaultManager.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	2/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Low

Recommendations

Remediation:

Initialize the batch pointer to `1` and seed `completedBatches[0]` with a terminal status (`SETTLED`) in `initialize`, then drop the dead `batchId == 0` branch from `finalizeBatch`. This removes the underflow by construction: `batchId - 1` is always a valid, previously-written slot.

Resolution:

Fixed in `993df50` :

The `initialize` function now sets `currentBatchId = 1` and pre-marks `completedBatches[0]` as `SETTLED`, eliminating the underflow on `completedBatches[batchId - 1]`. The dead-code `batchId == 0` special-case has been removed.

[F-2026-14842](#) - Missing tokenType Validation Allows Irreversible Token Invalidation - Info

Description:

The `updateTokenDetails` function allows an admin to modify the `AssetInfo` for an existing token, including setting `tokenType` to `AssetType.NONE`. This effectively invalidates the token, causing all deposit attempts with this token to fail, even though the token remains in the `tokenList` and appears to be supported.

The `updateTokenDetails` function does not validate that the new `tokenType` is not `NONE`:

```
function updateTokenDetails(uint16 _tokenId, AssetInfo calldata _asset) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(tokenList.contains(_asset.symbol));
    AssetInfo memory existing = assetInfo[_tokenId];
    require(existing.tokenType != AssetType.NONE, "OV-IVTA-01"); // Only checks EXISTING token
    require(existing.symbol == _asset.symbol, "OV-ITSN-01");
    assetInfo[_tokenId] = _asset; // ← No validation of _asset.tokenType!
}
```

The same is applicable to the `addTokenDetails` function.

When a user attempts to deposit, the `_depositTokens` function checks that `tokenType != NONE`:

```
function _depositTokens(...) internal {
    // ...
    for (uint256 i = 0; i < len; i++) {
        // ...
        AssetInfo memory asset = assetInfo[tokenId];
        require(asset.tokenType != AssetType.NONE, "OV-IVTA-01"); // ← Always fails if NONE
        // ...
    }
}
```

This leads to:

- **Token Becomes Unusable:** After setting `tokenType = NONE`, no deposits can be made with this token
- **Inconsistent State:** Token remains in `tokenList` (appears supported) but cannot be used
- **User Confusion:** Users see token as supported but transactions revert

- **No Recovery Path:** The same `updateTokenDetails` cannot restore the token because it requires `existing.tokenType != NONE`

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:**Fixed****Classification****Impact Rate:** 3/5**Likelihood Rate:** 1/5**Exploitability:** Dependent**Complexity:** Simple**Severity:** Info**Recommendations****Remediation:** Add validation to ensure the new `tokenType` is not `NONE`:

```
require(_asset.tokenType != AssetType.NONE, "OV-NTTZ-01");
```

Resolution: Fixed in `bce38c1`:

The issue as described does not exist in the current version of the **OmniVaultManager** contract `NONE` status has been removed.

[F-2026-14843](#) - Whitelisted Function Selector Mapping May Silently Overwrite Existing Entries - Info

Description:

The **OmniVaultExecutor** contract uses a `mapping(bytes4 => address)` to route function calls to whitelisted contracts based on function selectors.

The whitelisted function mapping stores a single contract address per function selector:

```
// bytes4 function signature => target contract, if address(0) then not white  
listed  
mapping(bytes4 => address) public whitelistedFunctions;  
function _setWhitelistedFunction(bytes4 _funcSignature, address _contract) internal {  
    require(trustedContracts[_contract] != ContractAccess.NONE, "OT-IVTC-01");  
    whitelistedFunctions[_funcSignature] = _contract; // ← Overwrites previous!  
}
```

The `fallback` function routes calls based on this mapping:

This one-to-one mapping design means that if a new function is whitelisted with a selector that already exists in the mapping, the previous entry will be silently overwritten.

Assets:

- contracts/vaults/OmniVaultExecutor.sol
(<https://github.com/Dexalot/contracts/commits/omnivaults/>)

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Consider adding a validation check in `_setWhitelistedFunction` to prevent accidental overwrites when the new contract address differs from the existing one:

```
function _setWhitelistedFunction(bytes4 _funcSignature, address _contract) internal {
    require(trustedContracts[_contract] != ContractAccess.NONE, "VE-IVTC-01");
    ;
    address existing = whitelistedFunctions[_funcSignature];
    require(existing == address(0) || existing == _contract, "VE-SEA0-01"); /
    / Selector already occupied
    whitelistedFunctions[_funcSignature] = _contract;
}
```

Alternatively, ensure that selector collision checks are part of the operational process when onboarding new trusted contracts or whitelisting new functions.

Resolution:

Fixed in `0c01b0c`:

A validation check to prevent accidental overwrites was added to `_setWhitelistedFunction`:

```
require(existing == address(0) || existing == _contract, "VE-SEA0-01");
```

[F-2026-14865](#) - Missing Event Emissions for Key State Changes in Vault Contracts - Info

Description:

Multiple administrative and state-changing functions across the OmniVault contract suite do not emit events, reducing transparency and making it difficult for off-chain systems to track configuration changes, monitor privileged operations, and maintain accurate audit trails.

The following functions perform significant state changes without emitting corresponding events:

OmniVaultExecutor.sol:

- `sendNative` - Native token transfers to trusted contracts
- `setWhitelistedFunction` - Individual function whitelist modifications
- `setWhitelistedFunctions` - Batch function whitelist modifications
- `setTrustedContract` - Trusted contract registry updates
- `setPortfolio` - Portfolio contract address changes

OmniVaultExecutorSub.sol:

- `setOmniVaultManager` - Manager contract address updates
- `setGasTopupAmount` - Gas topup threshold modifications
- `setFeeManager` - Fee manager address changes

OmniVaultManager.sol:

- `pauseVault` - Individual vault pause operations
- `unpauseVault` - Individual vault unpause operations
- `updateVaultDetails` - Vault configuration updates
- `addTokenDetails` - New token registration
- `updateTokenDetails` - Token configuration modifications
- `setPortfolio` - Portfolio contract address changes

OmniVaultCreator.sol:

- `withdrawCollectedFees` - Fee withdrawal operations

OmniVaultShare.sol:

- `setOmniVaultManager` - Manager contract address updates

This does not directly enable exploitation but significantly impacts operational transparency and monitoring capabilities:

- **Reduced transparency:** Off-chain monitoring systems cannot track administrative actions or configuration changes in real-time.

- **Incomplete audit trail:** Historical state changes cannot be reconstructed from event logs, complicating incident investigations.
- **Integration limitations:** Third-party systems and indexers relying on events for state synchronization will have incomplete or stale data.
- **Governance visibility:** Users and stakeholders cannot easily verify when critical parameters were modified.

Assets:

- contracts/vaults/OmniVaultManager.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]
- contracts/vaults/OmniVaultCreator.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]
- contracts/vaults/OmniVaultExecutor.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]
- contracts/vaults/OmniVaultExecutorSub.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]
- contracts/vaults/OmniVaultShare.sol
[https://github.com/Dexalot/contracts/commits/omnivaults/]

Status: Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Events should be emitted for all administrative and state-changing functions. Each event should include relevant parameters such as old and new values for configuration changes, the caller address, and any identifiers (vault IDs, token IDs) necessary for off-chain systems to accurately track state transitions. This aligns with best practices for transparent and auditable smart contract design.

Resolution: Mitigated in 80f46c4:

Event coverage was expanded across the vault suite for multiple administrative operations, including setTrustedContract, setPortfolio, whitelist updates, setOmniVaultManager, setGasTopupAmount, setFeeManager, withdrawCollectedFees, and OmniVaultShare setOmniVaultManager.

The listed **OmniVaultManager** administrative functions (`pauseVault` , `unpauseVault` , `updateVaultDetails` , `addTokenDetails` , `updateTokenDetails` , `setPortfolio`) still perform state changes without dedicated admin-configuration events, and **OmniVaultExecutor** `sendNative` still has no dedicated transfer event.

[F-2026-14870](#) - Missing NatSpec Documentation for Function Parameters in PortfolioSub - Info

Description:

In **PortfolioSub** contract, the `transferToken` and `safeIncrease` functions have parameters that are not documented in their NatSpec comments, reducing code readability and maintainability.

`transferToken`:

```
/**
 * @param _from Address of the sender
 * @param _to Address of the receiver
 * @param _symbol Symbol of the token
 * @param _quantity Amount of the token
 * @param _feeCharged Fee charged for the transaction
 * @param _transaction Transaction type
 * @param _decreaseTotalOnly If true, only total balance is decreased
 */
function transferToken(
    address _from,
    address _to,
    bytes32 _symbol,
    uint256 _quantity,
    uint256 _feeCharged,
    Tx _transaction,
    bool _decreaseTotalOnly,
    address _feeAddress
) private {
```

Missing: `@param _feeAddress`

`safeIncrease`:

```
/**
 * @notice Increases the balance of the user
 * @dev `_feeCharged` is deducted from the `_amount` before it is reflected in the user's balance
 * `_feeCharged` is transferred to feeAddress
 * @param _trader Address of the trader
 * @param _symbol Symbol of the token
 * @param _amount Amount of the token
 * @param _feeCharged Fee charged for the _transaction
 * @param _transaction Transaction type
 */
function safeIncrease(
```

```

    address _trader,
    bytes32 _symbol,
    uint256 _amount,
    uint256 _feeCharged,
    Tx _transaction,
    address _traderOther,
    address _feeAddress
  ) private {

```

Missing: `@param _traderOther`, `@param _feeAddress`

Third-party integrations may misuse undocumented parameters.
Developers unfamiliar with code may misunderstand parameter purpose.

Assets:

- contracts/PortfolioSub.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Add missing NatSpec documentation.

Resolution: Fixed in `41f29cd`:

NatSpec parameter coverage for both functions is complete in the current **PortfolioSub** implementation. `transferToken` documents `_feeAddress`, and `safeIncrease` documents both `_traderOther` and `_feeAddress`. The documentation gap described in the issue is no longer present.

F-2026-14872 - Undocumented Zero Parameters in

Description:

The `getNativeBridgeFee` function in **PortfolioMain** passes zero/empty values for multiple parameters without any explanation:

```
return this.getBridgeFee(_bridge, defaultChain, bytes32(0), 0, address(0), by
tes1(0));
//                                     ^^^^^^^^^^  ^  ^^^^^^^^^^  ^^
^^^^^^
//                                     _symbol    _quantity _sende
r _options
```

The function signature of `getBridgeFee` suggests these parameters should be meaningful:

```
function getBridgeFee(
    IPortfolioBridge.BridgeProvider _bridge,
    uint32 _dstChainListOrgChainId,
    bytes32 _symbol,           // Token symbol
    uint256 _quantity,        // Amount
    address _sender,          // Sender address
    bytes1 _options            // Transaction options
) external view returns (uint256 bridgeFee);
```

However, **PortfolioBridgeMain.getBridgeFee** ignores these parameters (they are unnamed in the implementation), which is not documented in the calling code.

This decreases the readability and maintainability of the codebase.

Assets:

- contracts/PortfolioMain.sol
- [<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add NatSpec comment explaining the design decision:

```
function getNativeBridgeFee(IPortfolioBridge.BridgeProvider _bridge) public view returns (uint256) {
    uint32 defaultChain = portfolioBridge.getDefaultDestinationChain();
    if (portfolioBridge.userPaysFee(defaultChain, _bridge)) {
        // symbol, quantity, sender, options are unused in PortfolioBridgeMain
        return this.getBridgeFee(_bridge, defaultChain, bytes32(0), 0, address(0), bytes1(0));
    }
    return 0;
}
```

Resolution: Fixed in [e99fc6e](#):

PortfolioMain. `getNativeBridgeFee` now includes an explicit inline comment stating that `symbol, quantity, sender, and options are unused in PortfolioBridgeMain`, and that empty placeholders are intentionally passed. The readability and maintainability concern described in the issue has been addressed through in-code documentation.

F-2026-14884 - receive Function in DexalotRouter Accepts ETH Without Processing, Leading to Stuck Funds - Info

Description:

The **DexalotRouter** contract implements an empty `receive` function that accepts native tokens (ETH/AVAX) without any processing or revert logic. Despite the NatSpec comment explicitly stating "`Not supported`," the function silently accepts any amount of native tokens sent directly to the contract. These funds become stuck and can only be recovered through admin intervention.

```
/**
 * @notice Not supported. To send native properly, the call must be forwarded
 * to an allowed MainnetRFQ contract via partialSwap or simpleSwap.
 */
receive() external payable {}
```

The current implementation:

- Accepts any amount of ETH silently
- Does not emit any events
- Does not revert or return funds
- Contradicts its own documentation ("`Not supported`")

Assets:

- contracts/DexalotRouter.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	2/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

If the contract doesn't need to receive ETH outside of payable functions with calldata, remove `receive` completely.

Resolution:

Fixed in `219781c`:

DexalotRouter. `receive` has been removed, eliminating the direct-transfer sink condition described in the issue.

[F-2026-14885](#) - Missing Reentrancy Guards on Swap Functions Reduces Defense-in-Depth - Info

Description:

The **DexalotRFQ** contract only applies the `nonReentrant` modifier to `removeFromSwapQueue`, while the primary swap functions `simpleSwap`, `partialSwap`, `xChainSwap`, and `processXFerPayload` lack this protection. Although the nonce-marking-before-execution pattern provides protection against same-swap reentrancy, the absence of explicit guards reduces defense-in-depth against potential cross-function reentrancy vectors

The swap functions mark nonces as used before external calls, which prevents the same swap from being executed twice:

```
// Nonce marked before execution
usedNonces[bucket] = usedNonces[bucket] | mask;
// ... later performs external calls
```

However, during `_releaseFunds`, native token transfers to the `destTrader` address occur:

```
function _releaseFunds(SwapData memory _swapData) private {
    address destTrader = UtilsLibrary.bytes32ToAddress(_swapData.destTrader);
    // ...
    (bool success, ) = destTrader.call{value: _swapData.destAmount}("");
    // ...
}
```

A malicious `destTrader` contract receiving native tokens could potentially re-enter the **DexalotRFQ** contract and call other functions during execution. While the nonce protection prevents re-executing the same swap, cross-function reentrancy into other swap functions or state-modifying operations is not explicitly blocked.

Assets:

- contracts/DexalotRFQ.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: The `nonReentrant` modifier should be added to `simpleSwap`, `partialSwap`, `xChainSwap`, and `processXferPayload` functions to provide explicit reentrancy protection. This follows the defense-in-depth principle and ensures protection remains effective even if future code modifications alter the execution flow or nonce-checking logic.

Resolution: Fixed in `4d94a89`:
`nonReentrant` was added to `simpleSwap`, `partialSwap`, `xChainSwap`, and `processXferPayload` in **DexalotRFQ**.

[F-2026-14886](#) - Missing Reentrancy Guard in multiPartialSwap and fallback Functions - Info

Description:

The **DexalotRouter** contract makes multiple external calls to RFQ contracts without implementing reentrancy protection. While the risk is mitigated by admin-controlled allowlists, the absence of a reentrancy guard leaves the contract vulnerable if any allowed RFQ contract is compromised or contains vulnerabilities.

In `multiPartialSwap`:

```
function multiPartialSwap(
    IDexalotRFQ.Order calldata _orderA,
    bytes calldata _signatureA,
    uint256 _takerAmountA,
    IDexalotRFQ.Order calldata _orderB,
    bytes calldata _signatureB
) external payable {
    // ... validations ...

    uint256 preBal = _orderA.makerAsset == address(0)
        ? address(this).balance
        : IERC20(_orderA.makerAsset).balanceOf(address(this));
    // ... token transfer ...
    // FIRST EXTERNAL CALL - potential reentrancy point
    (bool success, ) = payable(_orderA.maker).call{value: msg.value}(_signatureA);
    require(success, "DR-PSA-01");
    // STATE READS AFTER EXTERNAL CALL
    uint256 takerAmountB;
    bool isHopNative = (_orderA.makerAsset == address(0));
    if (!isHopNative) {
        IERC20 bInputAsset = IERC20(_orderA.makerAsset);
        takerAmountB = bInputAsset.balanceOf(address(this)) - preBal; // Balance read
        bInputAsset.safeTransfer(_orderB.maker, takerAmountB);
    } else {
        takerAmountB = address(this).balance + msg.value - preBal; // Balance read
    }
    // SECOND EXTERNAL CALL
    (success, ) = payable(_orderB.maker).call{value: isHopNative ? takerAmountB : 0}(_signatureB);
    require(success, "DR-PSB-01");
}
```

In `fallback`:

```
fallback() external payable {
    // ... validations and setup ...

    if (takerAsset != address(0)) {
        IERC20(takerAsset).safeTransferFrom(msg.sender, targetImplementation,
amount);
    }
    // EXTERNAL CALL - potential reentrancy point
    (bool success, ) = targetImplementation.call{value: msg.value}(newCallData
);

    // ... return handling ...
}
```

The contract follows a pattern vulnerable to reentrancy:

1. Read state (`preBal = balance`)
2. External call `to _orderA.maker`
3. Read state (current balance)
4. Calculate difference
5. External call to `_orderB.maker`

This leads to the following risks:

1. Future Changes: New RFQ contracts added to allowlist might have vulnerabilities
2. Compromised RFQ: If an allowed RFQ is compromised, reentrancy becomes exploitable
3. Upgradeable Contracts: If RFQ contracts are upgradeable, malicious upgrades could exploit this

Assets:

- `contracts/DexalotRouter.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Dependent

Complexity: Simple

Severity:

Info

Recommendations

Remediation: Consider adding OpenZeppelin's `ReentrancyGuard` protection.

Resolution: Mitigated in `1c75fe5`:
`multiPartialSwap` now includes `nonReentrant`, reducing reentrancy exposure on the multi-call path. The `fallback` forwarding path remains without a reentrancy modifier, while access is constrained by selector checks and RFQ allowlisting. Risk is reduced but not fully eliminated at the router entry-point level.

[F-2026-14894](#) - Missing Explicit Check for Maximum Token Limit

Results in Overflow Revert - Info

Description:

The `addRewardToken` function does not include an explicit check for the maximum token capacity. Since `allTokens` is a `uint32` bitmap that can only track 32 tokens, adding a 33rd token causes Solidity's checked arithmetic to revert with a panic error rather than a descriptive error message.

The `addRewardToken` function calculates the next available token ID using bit manipulation:

```
function addRewardToken(bytes32 _symbol) external whenPaused onlyOwner {
    uint32 tokenId = ~allTokens & (allTokens + 1);
    tokens[tokenId] = _symbol;
    allTokens |= tokenId;
    emit AddRewardToken(_symbol, tokenId, block.timestamp);
}
```

When all 32 bits are set and a 33rd token is added:

- `allTokens + 1` triggers an arithmetic overflow
- Solidity 0.8.x's checked arithmetic reverts with `Panic(0x11)`

While the overflow is caught and the transaction reverts, the error provides no indication that the maximum token limit has been reached.

Assets:

- `contracts/token/IncentiveDistributor.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Add explicit bounds check in `addRewardToken`:

```
require(allTokens != type(uint32).max, "ID-MAXT-01"); // Max 32 tokens
```

Resolution:

Fixed in `3ca640`:

`daddRewardToken` now includes an explicit capacity guard:

```
require(allTokens != type(uint32).max, "ID-MAXT-01");
```

This prevents the overflow path at full bitmap capacity and replaces generic panic behavior with a descriptive revert reason.

F-2026-14895 - Missing Validation for Duplicate and Empty Token Symbols - Info

Description:

The `initialize` and `addRewardToken` functions do not validate that the provided symbol is non-empty or unique. This allows the same symbol to be registered multiple times under different token IDs, and permits `bytes32(0)` to be added as a reward token, potentially causing claim failures.

Both functions add tokens without validation:

```
function initialize(bytes32 _alotSymbol, address __signer, address __portfolio) public initializer {
    // ...
    uint32 tokenId = ~allTokens & (allTokens + 1);
    tokens[tokenId] = _alotSymbol; // No check for bytes32(0) or duplicates
    allTokens |= tokenId;
    // ...
}

function addRewardToken(bytes32 _symbol) external whenPaused onlyOwner {
    uint32 tokenId = ~allTokens & (allTokens + 1);
    tokens[tokenId] = _symbol; // No check for bytes32(0) or duplicates
    allTokens |= tokenId;
    emit AddRewardToken(_symbol, tokenId, block.timestamp);
}
```

- If an administrator adds "ALOT" twice, it occupies two token ID slots in the bitmap. Claims referencing the duplicate symbol would depend on which token ID is used in the signature, creating ambiguity. The off-chain signer must track which token ID corresponds to which symbol, increasing complexity and error potential.
- Adding `bytes32(0)` as a reward token creates a token entry that maps to an empty symbol. When claim calls `portfolio.bulkTransferTokens` with an empty symbol, the transfer may fail or behave unexpectedly.

Assets:

- `contracts/token/IncentiveDistributor.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Validation checks should be added to both `initialize` and `addRewardToken` to ensure the symbol is non-empty and not already registered.

Resolution: Fixed in `4d57fce`:

`initialize` now validates `_alotSymbol` is non-empty and present in **Portfolio** token details.

`addRewardToken` now validates non-empty/valid symbol and rejects duplicates via explicit iteration over existing token slots.

F-2026-14897 - Missing Event Emissions for State-Changing Functions - Info

Description:

Multiple administrative and state-changing functions across **IncentiveDistributor**, **DexalotRFQ**, and **DexalotRouter** contracts do not emit events, reducing transparency and making it difficult for off-chain systems to track configuration changes, monitor privileged operations, and maintain accurate audit trails.

The following functions perform significant state changes without emitting corresponding events:

IncentiveDistributor.sol:

- `retrieveRewardToken` - Retrieval of unclaimed reward tokens to owner
- `updateSigner` - Reward signer address updates

DexalotRFQ.sol:

- `receive` - Native token deposits from rebalancers
- `sendTokenBalance` - ERC20 token deposits for liquidity
- `setSlippagePoints` - Slippage configuration updates

DexalotRouter.sol:

- `retrieveToken` - Native and ERC20 token retrieval by admin
- `multiPartialSwap` - Multi-hop swap execution

This does not directly enable exploitation but significantly impacts operational transparency and monitoring capabilities.

Assets:

- `contracts/DexalotRFQ.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]
- `contracts/DexalotRouter.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]
- `contracts/token/IncentiveDistributor.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Events should be emitted for all administrative and state-changing functions, including relevant parameters such as old and new values for configuration changes, amounts for token operations, and caller addresses for privileged actions.

Resolution: Mitigated in [6c280d2](#):

Event coverage was improved for part of the reported scope:

- **IncentiveDistributor:** events were added for `retrieveRewardToken` and `updateSigner`.
- **DexalotRFQ:** event emission was added for `setSlippagePoints`.

Full remediation was not achieved for all reported functions:

- **DexalotRFQ:** `sendTokenBalance` and `receive` still do not emit dedicated events.
- **DexalotRouter:** `retrieveToken` and `multiPartialSwap` still do not emit dedicated events.

[F-2026-14899](#) - Misleading Documentation and Dead Code in floor Function - Info

Description:

The `floor` function in **UtilsLibrary** contains NatSpec documentation that describes behavior the code does not implement, along with commented-out code that would enforce a minimum return value. This creates confusion about the intended functionality.

The function documentation states:

```
/**
 * @notice Round down a uint256 value. Used for the fees to avoid dust.
 * @dev example: a = 1245, m: 2 ==> 1200. But always take a min fee
 * a = 1, m : 2 ==> 100 instead of flooring to 0
 * @param _a number to round down
 * @param _m number of digits from the right to round down
 * @return uint256 .
 */
function floor(uint256 _a, uint256 _m) internal pure returns (uint256) {
    return (_a / 10 ** _m) * 10 ** _m;
    // if (_a == 0) {
    //     return 0;
    // }
    // uint256 n = (_a / 10 ** _m) * 10 ** _m;
    // if (n == 0) {
    //     return 10 ** _m;
    // }
    // return n;
}
```

The documentation claims "always take a min fee" and gives an example where `floor(1, 2)` returns `100`. However, the actual implementation returns `0` for this input since $1 / 100 = 0$ and $0 * 100 = 0$.

The commented-out code would implement the documented minimum fee behavior by returning 10^m when the result would otherwise be zero (for non-zero inputs). This suggests an abandoned design change that was never completed or was intentionally reverted.

Assets:

- `contracts/library/UtilsLibrary.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:	Either update the documentation to accurately reflect the current implementation (simple floor without minimum), or uncomment and activate the minimum fee logic if that behavior is intended. The commented-out code should be removed if no longer needed to reduce confusion.
Resolution:	Fixed in <code>bce03d3</code>: UtilsLibrary <code>floor</code> documentation was aligned with the implemented behavior, and obsolete commented-out minimum-fee logic was removed. The function now has consistent NatSpec and code semantics: simple floor rounding without enforced minimum return.

[F-2026-14904](#) - Missing Validation in `setDisplayDecimals` Can Cause DoS for Trade Pair - Info

Description:

The `setDisplayDecimals` function lacks validation to ensure that `_displayDecimals` does not exceed the token's EVM decimals. Setting `displayDecimals > decimals` causes an arithmetic underflow in the `decimalsOk` function, which reverts all order submissions for that trade pair.

```
function setDisplayDecimals(
    bytes32 _tradePairId,
    uint8 _displayDecimals,
    bool _isBase
) external override onlyRole(DEFAULT_ADMIN_ROLE) {
    TradePair storage tradePair = tradePairMap[_tradePairId];
    uint256 oldValue = tradePair.baseDisplayDecimals;
    if (_isBase) {
        tradePair.baseDisplayDecimals = _displayDecimals;
    } else {
        oldValue = tradePair.quoteDisplayDecimals;
        tradePair.quoteDisplayDecimals = _displayDecimals;
    }
    emit ParameterUpdated(PARAMETER_UPDATED_VERSION, _tradePairId, "T-DIS
PDEC", oldValue, _displayDecimals);
}
```

The `decimalsOk` function in **UtilsLibrary** performs arithmetic that requires `_decimals >= _displayDecimals`:

```
function decimalsOk(uint256 _value, uint8 _decimals, uint8 _displayDecimals)
internal pure returns (bool) {
    return (_value - (_value - ((_value % 10 ** _decimals) % 10 ** (_decimals - _displayDecimals)))) == 0;
}
```

Critical expression: `10 ** (_decimals - _displayDecimals)`. When `_displayDecimals > _decimals`:

- `_decimals - _displayDecimals` results in arithmetic underflow
- In Solidity 0.8+, this causes an automatic revert

The `decimalsOk` function is called in `addOrderChecks` and in `setAuctionPrice`.

Assets:

- contracts/TradePairs.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Mitigated

Classification**Impact Rate:**

2/5

Likelihood Rate:

1/5

Exploitability:

Dependent

Complexity:

Simple

Severity:

Info

Recommendations**Remediation:**Add validations in `setDisplayDecimals` and `addTradePair`.**Resolution:**

The issue has been mitigated through off-chain validation in deployment and setup scripts rather than implementing an on-chain require check.

Client Response:

An explicit require check was intentionally omitted from `setDisplayDecimals` + `addTradePair` as `TradePairs.sol` is near the maximum size capacity. We mitigate this with our deployment/setup scripts which have explicit checks to ensure `displayDecimals ≤ decimals` always holds, this is done in both our database upsert and hardhat scripts.

[F-2026-14912](#) - Inconsistent Documentation for Default Allowed Slippage Percent - Info

Description:

The default `allowedSlippagePercent` was changed from 20% to 3% in the code, but comments in multiple locations still reference the old 20% default value, creating documentation inconsistency.

The actual default value set in `addTradePair`:

```
// with default allowedSlippagePercent of 20, the market orders cannot be filled
// worst than 80% of the bestBid and 120% of bestAsk
tradePair.allowedSlippagePercent = 3; // (3% = 3/100)
```

The comment above the assignment still describes behavior for 20% slippage ("80% of bestBid and 120% of bestAsk"), while the actual value is set to 3%.

The NatSpec for `setAllowedSlippagePercent` also references the old default:

```
* @param _allowedSlippagePercent allowed slippage percent=20 (Default = 20
: 20% = 20/100)
```

Comments suggest 20% slippage tolerance while actual behavior is much stricter at 3%.

Assets:

- `contracts/TradePairs.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Update all comments and NatSpec documentation to reflect the current 3% default value. The inline comment should describe behavior as "97% of bestBid and 103% of bestAsk" to match the actual slippage tolerance, and the NatSpec should indicate "Default = 3 : 3% = 3/100".

Resolution:

Fixed in 43f1202:

Comments and NatSpec references for `allowedSlippagePercent` were updated to match the implemented 3% default, including the implied execution bounds (97% of best bid and 103% of best ask).

[F-2026-14915](#) - Native Tokens Lost During ERC20 Swaps Through Router - Info

Description: **DexalotRouter** always forwards `msg.value` to **DexalotRFQ** regardless of whether the swap involves native tokens or ERC20 tokens. **DexalotRFQ** only refunds excess native tokens when `srcAsset == address(0)` (native swap), not for ERC20 swaps.

DexalotRouter - `fallback`:

```
(bool success, ) = targetImplementation.call{value: msg.value}(newCallData);
```

DexalotRouter - `multiPartialSwap`:

```
(bool success, ) = payable(_orderA.maker).call{value: msg.value}(callDataA);
```

DexalotRFQ - `_takeFunds`:

```
function _takeFunds(SwapData memory _swapData) private {
    if (_swapData.srcAsset == address(0)) {
        require(msg.value >= _swapData.srcAmount, "RF-IMV-01");
        if (keepWrapped) {
            IWrappedToken(wrappedNative).deposit{value: _swapData.srcAmount}();
        }
        return; // Only handles native tokens
    }
    // For ERC20: msg.value is NOT checked or refunded!
    if (_swapData.isDirect) {
        IERC20(_swapData.srcAsset).safeTransferFrom(_swapData.msgSender, address(this), _swapData.srcAmount);
    }
    // ...
}
```

DexalotRFQ - `_refundNative`:

```
function _refundNative(SwapData memory _swapData) private {
    // Only refunds for native swaps!
    if (_swapData.srcAsset == address(0) && msg.value > _swapData.srcAmount) {
        (bool success, ) = payable(_swapData.msgSender).call{value: msg.value - _swapData.srcAmount}("");
        require(success, "RF-TF-02");
    }
}
```

```
}  
}
```

This affects both `simpleSwap` and `partialSwap` through **DexalotRouter** and `multiPartialSwap` function directly and may lead to:

- Direct fund loss for any user who sends native tokens with ERC20 swaps
- Users cannot recover lost funds (no rescue function for arbitrary users)

Assets:

- `contracts/DexalotRFQ.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]
- `contracts/DexalotRouter.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	3/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Implement validation in **DexalotRouter**:

```
if (takerAsset != address(0)) {  
    // ERC20 swap - should not have msg.value  
    require(msg.value == 0, "DR-NFES-01"); // No funds for ERC20 swaps  
    IERC20(takerAsset).safeTransferFrom(msg.sender, targetImplementation,  
amount);  
    (bool success, ) = targetImplementation.call(newCallData);  
}
```

Or restore the check from **MainnetRFQ**:

```
require(msg.value == 0, "RF-NSIV-01");
```

Resolution:

Fixed in [8c5096a](#):

DexalotRouter now enforces zero native value for ERC20-input swaps before forwarding calls. In both entry paths, ERC20 branches include `require(msg.value == 0, "DR-NFES-01")`, preventing accidental native-asset forwarding during ERC20 swaps.

```
require(msg.value == 0, "DR-NFES-01");
```

F-2026-14919 - Inconsistent Native Token Handling in sendTokenBalance Function - Info

Description: The `sendTokenBalance` function is marked as `payable`, allowing it to receive native tokens (ETH/AVAX), but its logic completely ignores `msg.value`.

```
function sendTokenBalance(address _asset, uint256 _amount) external payable
onlyRole(REBALANCER_ADMIN_ROLE) {
    require(_asset != address(0), "RF-NSNA-01");
    IERC20(_asset).safeTransferFrom(msg.sender, address(this), _amount);
}
```

This leads to an inconsistent approach:

- Marked as payable (can accept ETH)
- But prohibits `address(0)` (native tokens)
- Lacks `require(msg.value == 0)` protection against accidental ETH sends

This creates an asymmetric API where:

- Withdrawing (`claimBalance`): supports both native and ERC20 tokens
- Depositing (`sendTokenBalance`): only supports ERC20, but accepts native tokens silently

Rebalancer may accidentally send native tokens along with an ERC20 transaction, and those tokens will be locked in the contract with no way to attribute them to the deposit

Assets:

- contracts/DexalotRFQ.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Remove `payable` modifier since function is intended only for ERC20.
- Or support both asset types for consistency with `claimBalance`.

Resolution:

Fixed in `229271a`:

`sendTokenBalance` is no longer payable in the current implementation. The function accepts only ERC20 asset transfers and rejects native-asset path by design through `_asset != address(0)`. The silent native-asset acceptance condition described in the issue is not present.

F-2026-14931 - Redundant Conditional Logic in decrement Function

- Info

Description:

The conditional logic for updating `userProvidedLiquidity` is redundant. The delete operation on a mapping entry is equivalent to setting it to the default value (`0` for `uint256`).

```
if (userLiquidity > 0) {
    userProvidedLiquidity[_withdrawal.symbolId][_withdrawal.traderaddress] =
    userLiquidity;
} else {
    // clean the state if user liquidity provided is 0
    delete (userProvidedLiquidity[_withdrawal.symbolId][_withdrawal.traderadd
    ress]);
}
```

Since assigning `0` to a mapping entry has the same effect as delete, the entire if/else block can be replaced with a single assignment.

Assets:

- contracts/InventoryManager.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Simplify to:

```
userProvidedLiquidity[_withdrawal.symbolId][_withdrawal.traderaddress] = user
Liquidity;
```

Resolution:

Fixed in [7cfbc74](#):

Current **InventoryManager**. `decrement` no longer contains the `if/else` branch with explicit delete for `userProvidedLiquidity`. The implementation updates state only when `userLiquidity > 0` and performs a single assignment after reduction. The redundant delete-path pattern described in the issue is not present in the current code.

F-2026-14932 - Gas Inefficiency in powMultipleOf4BPS - Info

Description:

The `powMultipleOf4BPS` function always computes `R2`, `R4`, `R8`, `R16` regardless of which `K` value is needed:

```
function powMultipleOf4BPS(uint256 base, uint256 K) private pure returns (uint256 result_) {
    uint256 R = base;
    // ALWAYS computed, even if not needed:
    uint256 R2 = mulBPS(R, R); // ← wasted if K=8
    uint256 R4 = mulBPS(R2, R2); // ← wasted if K=8
    uint256 R8 = mulBPS(R4, R4); // ← always needed
    uint256 R16 = mulBPS(R8, R8); // ← wasted if K=8,12
    if (K == 8) {
        return R8; // R16 was computed but never used!
    }
    // ...
}
```

For `K=8`, ~40% of computations are wasted.

K	Needed	Computed	Wasted
8	R2, R4, R8	R2, R4, R8, R16	R16
12	R2, R4, R8	R2, R4, R8, R16	R16
16	R2, R4, R8, R16	R2, R4, R8, R16	None

Assets:

- contracts/library/InventoryFeeCalculatorLibrary.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation:

Compute lazily:

```
function powMultipleOf4BPS(uint256 base, uint256 K) private pure returns (uint256) {
    uint256 R2 = mulBPS(base, base);
    uint256 R4 = mulBPS(R2, R2);
    uint256 R8 = mulBPS(R4, R4);

    if (K == 8) return R8;
    if (K == 12) return mulBPS(R8, R4);

    uint256 R16 = mulBPS(R8, R8); // Only compute if needed

    if (K == 16) return R16;
    // ...
}
```

Resolution:

Fixed in [29e009c](#):

`powMultipleOf4BPS` now applies lazy computation for higher powers. Current implementation computes `R16` only after handling `K == 8` and `K == 12`, removing unnecessary multiplication for those exponent paths. The efficiency issue described in the report is no longer present.

[F-2026-14933](#) - Misleading NatSpec Comment in powMultipleOf4BPS

Function - Info

Description:

The NatSpec documentation for the `powMultipleOf4BPS` function contains a misleading description that conflates two unrelated concepts: the exponent value `K` and Basis Points (BPS).

```
/**
 * @notice Efficiently computes base^K where K is a multiple of 4 BPS (e.g.,
 * 8, 12, 16, ..., 32)
 * @param base The base value
 * @param K The exponent value (must be a multiple of 4)
 */
```

The phrase "`multiple of 4 BPS`" is incorrect and confusing because:

1. `BPS` (Basis Points) = 10000 — This is a precision scalar used for fixed-point arithmetic in fee calculations
2. `K` values (8, 12, 16, 20, 24, 28, 32) — These are simple integer exponents, multiples of 4

The comment suggests that `K` should be " $4 \times 10000 = 40000$ " or similar, which is not the case. The actual requirement is simply that `K` must be a multiple of 4 (the integer 4, not 4 BPS).

Assets:

- `contracts/library/InventoryFeeCalculatorLibrary.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

Update the NatSpec comment to accurately describe the `K` parameter.

Resolution:

Fixed in `2872cc0`:

NatSpec for `powMultipleOf4BPS` now states that `K` is "a multiple of 4" and no longer describes `K` as "multiple of 4 BPS." Current parameter description correctly distinguishes exponent semantics from basis-point scaling.

F-2026-14940 - Excess Native Token Not Refunded When User Pays Bridge Fee - Info

Description:

When `userPaysFee[destinationChainId][bridge]` is set to `true`, users are required to pay the bridge fee by sending native tokens (`msg.value`) with their transaction. The contract validates that the user sent at least the required fee amount using `require(msg.value >= fee)`, but it does not refund any excess amount sent above the required fee.

```
if (isUserFeePayer) {  
    require(_userFeePayer != address(0), "PB-UFPE-01");  
    require(msg.value >= fee, "PB-IUMF-01");  
}
```

In contrast, when `isUserFeePayer = false` (protocol pays), the contract correctly refunds any native tokens the user may have accidentally sent:

```
else {  
    if (_userFeePayer != address(0)) {  
        (bool success, ) = _userFeePayer.call{value: msg.value}("");  
        require(success, "PB-UFPR-01");  
        require(address(this).balance > fee, "PB-CBIZ-01");  
    }  
    _userFeePayer = address(this);  
}
```

This inconsistent behavior means that when users are responsible for paying the bridge fee, any excess native tokens they send become lost for them.

Assets:

- contracts/PortfolioBridgeMain.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Add refund logic for excess native tokens when `isUserFeePayer == true`.

Resolution: Fixed in `c6682ae`:

Excess native assets are handled in the user-fee path. Current logic validates `msg.value >= fee`, computes `feeOverpay`, and refunds overpayment.

```
uint256 feeOverpay = msg.value - fee;
// LZ handles refund within bridgeApp
if (_bridge != BridgeProvider.LZ && feeOverpay > 0) {
    (bool success, ) = _userFeePayer.call{value: feeOverpay}("");
    require(success, "PB-UFPR-02");
}
```

[F-2026-14941](#) - Off-by-One Error in PortfolioBridgeMain Contract Balance Check - Info

Description: In the `sendXChainMessageInternal` function, when the protocol (not the user) pays the bridge fee, there is an off-by-one error in the balance validation check:

```
require(address(this).balance > fee, "PB-CBIZ-01");
```

The condition uses strict greater-than (`>`) instead of greater-than-or-equal (`>=`).

```
require(address(this).balance > fee, "PB-CBIZ-01");
```

When the contract's balance is exactly equal to the required fee, the transaction will revert with error `PB-CBIZ-01`, even though the contract has sufficient funds to pay the fee.

Assets:

- contracts/PortfolioBridgeMain.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Change the strict greater-than operator to greater-than-or-equal:

```
require(address(this).balance >= fee, "PB-CBIZ-01");
```

Resolution: Fixed in `b9f1e48`:

The balance validation in `sendXChainMessageInternal` uses `require(address(this).balance >= fee, "PB-CBIZ-01");`. Exact-balance fee payments are accepted, and reversion occurs only when available funds are below the required fee. The off-by-one condition described in the issue is no longer present.

[F-2026-14985](#) - Inverted NatSpec Comment in _updateTakerFeeCollector - Info

Description: The NatSpec comment in the **PortfolioSubHelper** describes the logic inversely compared to the actual code implementation and requirements:

```
@dev If the maker address is same as the taker fee collector address for the  
tradePair,  
then return address(0) to indicate default behavior (fee goes to feeTreasury)
```

Actual code logic:

```
function _updateTakerFeeCollector(  
    bytes32 _tradePairId,  
    address _makerAddr  
) internal view returns (address takerFeeCollector) {  
    address collector = takerFeeCollectors[_tradePairId];  
    if (collector != _makerAddr) {  
        takerFeeCollector = address(0);  
    } else {  
        takerFeeCollector = collector;  
    }  
}
```

Developers reading the documentation will gain an incorrect understanding of the logic. This may also lead to errors during integration or code modification.

Assets:

- contracts/PortfolioSubHelper.sol
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation:

Correct the comment to accurately describe the behavior:

```
/**
 * @dev Returns the taker fee collector only when maker is the designated
 * collector
 * for this tradepair; otherwise returns address(0) for default treasury behav
 * ior.
 */
```

Resolution:

Fixed in [dea59c6](#):

NatSpec for `_updateTakerFeeCollector` now matches runtime behavior.

Current documentation states that the taker fee collector is returned only when the maker equals the configured collector for the trade pair; otherwise `address(0)` is returned for default treasury routing.

Implementation at **PortfolioSubHelper**. `_updateTakerFeeCollector` follows the same condition (`collector != _makerAddr` $\Rightarrow$ `address(0)` , else `collector`).

[F-2026-16205](#) - [DualDefense] Duplicate-Heavy Deposit Batches Are Atomic, Cannot Be Partially Settled, and Block Follower Batches - Info

Description: `requestDeposit` accepts arbitrarily long duplicate token arrays and only limits batches by request count. Once such a batch is finalized, `bulkSettleState()` must replay the full batch hash in one transaction.

The protocol offers no partial settlement path and no unwind path for an already finalized batch by batch id. As a result, an oversized finalized batch can block follower batches in FIFO order.

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Enforce per-request input sanitization so each deposit contributes bounded, predictable work to a batch. Specifically, in `_depositTokens` (and symmetrically in any other path that accepts a `_tokens / _amounts` pair), reject duplicate `tokenIds` and arrays longer than the vault's allowed token set.

Resolution: Fixed in `993df50` :

The `_depositTokens` function now calls `_checkDuplicateTokenIds`, reverting if any `tokenId` appears more than once. Without duplicates, per-request work is bounded to the unique token count per vault (2–5), keeping batches settleable within block gas limits.

[F-2026-16206](#) - [DualDefense] Current Design Allows for maxPerDeposit To Be Bypassed via Same tokenIds in requestDeposit - Info

Description: The `minPerDeposit` / `maxPerDeposit` bounds are enforced per array entry, not per tokenId aggregated across the request. A user can repeat the same `tokenId` multiple times and split a large deposit into chunks, each individually $\leq \text{maxPerDeposit}$, while the total deposited for that token in a single request exceeds `maxPerDeposit`.

If `maxPerDeposit` is intended to cap exposure, compliance limits, or risk per token per deposit request, it is not enforceable. Users can deposit more than allowed by providing duplicate `tokenIds` (and corresponding duplicate `symbols` to `PortfolioSub.bulkTransferTokens`).

Assets:

- `contracts/vaults/OmniVaultManager.sol`
[<https://github.com/Dexalot/contracts/commits/omnivaults/>]

Status:

Fixed

Classification

Impact Rate: 1/5
Likelihood Rate: 1/5
Exploitability: Independent
Complexity: Simple
Severity: Info

Recommendations

Remediation: Enforce per-request input sanitization so each deposit contributes bounded, predictable work to a batch. Specifically, in `_depositTokens` (and symmetrically in any other path that accepts a `_tokens` / `_amounts` pair), reject duplicate `tokenIds` and arrays longer than the vault's allowed token set.

Resolution: Fixed in `993df50` :

The `_depositTokens` function now calls `_checkDuplicateTokenIds` before the per-entry validation loop, reverting on repeated `tokenId` values. Each token can

appear at most once, so the per-entry `maxPerDeposit` check effectively enforces the per-token cap.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Dexalot/contracts/
Commit	396dddee7c99c809b9599bec94c42823306abbcd
Final Commit	993df50c014f3c8a1a33e45b4364ab9e709662fb
Whitepaper	https://dexalot.com/docs/DEXALOT-Litepaper.pdf
Requirements	https://docs.dexalot.com/en/contracts/ ; NatSpec
Technical Requirements	https://docs.dexalot.com/en/contracts/ ; NatSpec

Asset	Type
contracts/bridgeApps/DefaultBridgeApp.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/bridgeApps/ICMApp.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/DexalotRFQ.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/DexalotRouter.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/InventoryManager.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/library/InventoryFeeCalculatorLibrary.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/library/UtilsLibrary.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/OrderBooks.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/PortfolioBridgeMain.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/PortfolioBridgeSub.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/PortfolioMain.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/PortfolioSub.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/PortfolioSubHelper.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/token/IncentiveDistributor.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract

Asset	Type
contracts/TradePairs.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/vaults/OmniVaultCreator.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/vaults/OmniVaultExecutor.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/vaults/OmniVaultExecutorSub.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/vaults/OmniVaultManager.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract
contracts/vaults/OmniVaultShare.sol [https://github.com/Dexalot/contracts/commits/omnivaults/]	Smart Contract

Appendix 3. Additional Valuables

Verification of System Invariants

During the audit of Dexalot, Hacken followed its methodology by performing fuzz-testing on the project's main functions. Foundry, a tool used for fuzz-testing, was employed to check how the protocol behaves under various inputs. Due to the complex and dynamic interactions within the protocol, unexpected edge cases might arise. Therefore, it was important to use fuzz-testing to ensure that several system invariants hold true in all situations.

Fuzz-testing allows the input of many random data points into the system, helping to identify issues that regular testing might miss. A specific Foundry fuzzing suite was prepared for this task, and throughout the assessment, 22 invariants were tested over 256 runs. This thorough testing ensured that the system works correctly even with unexpected or unusual inputs.

Invariant	Test Result	Run Count
INV-1 <code>invariant_kBounds</code> — K within [8, 32]	Passed	256
INV-2 <code>invariant_futureKTimeLock</code> — <code>futureKTime</code> respects 1h minimum	Passed	256
INV-3 <code>invariant_inventoryAccounting</code> — ghost inventory matches on-chain	Passed	256
INV-4 <code>invariant_totalInventoryConsistency</code> — ghost inventory matches on-chain	Passed	256
INV-5 <code>handler_updateKEarly</code> — <code>updateK</code> reverts before <code>futureKTime</code>	Passed	256
INV-6 <code>handler_decrementOverflow</code> — decrement reverts on underflow	Passed	256
INV-7 <code>handler_remove</code> — remove succeeds iff <code>inventory == 0</code>	Passed	256
INV-8 <code>handler_calculateWithdrawalFee</code> — <code>fee == 0</code> single chain	Passed	256
INV-9 <code>handler_calculateWithdrawalFee</code> — <code>fee == 0</code> empty chain	Passed	256
INV-10 <code>handler_calculateWithdrawalFee</code> — <code>fee == 0</code> user liquidity covers qty	Passed	256
INV-11 <code>handler_calculateWithdrawalFee</code> — <code>fee <= 5.01%</code> of effective qty	Passed	256
INV-12 <code>handler_aclBridgeRevert</code> — bridge functions revert without role	Passed	256
INV-13 <code>handler_aclAdminRevert</code> — admin functions revert without role	Passed	256
INV-14 <code>handler_decrementFullLiquidity</code> — liquidity slot deleted at zero	Passed	256
INV-15 <code>handler_setScalingFactorsInvalid</code> — revert on unregistered token	Passed	256
INV-16 <code>handler_updatePortfolioBridgeSub</code> — role transfer on bridge swap	Passed	256
INV-17 <code>handler_verifyDefaultRatio</code> — default ratio is BPS/numChains	Passed	256
INV-18 <code>invariant_kMultipleOf4</code> — K multiple of 4	Failed	256
INV-19 <code>invariant_userLiquidityAccounting</code> — ghost liquidity matches on-chain	Passed	256
INV-20 <code>invariant_futureKBounds</code> — <code>futureK</code> within [8, 32]	Passed	256
INV-21 <code>invariant_bridgeNonZero</code> — bridge address never zero	Passed	256
INV-22 <code>invariant_bridgeHasRole</code> — current bridge holds its role	Passed	256

Additional Recommendations



The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.